Introduction to Computing (R & Python) and LaTeX

Command-line skills, data analysis in R and Python, and professional documents in LaTeX.

My courses



18 lessons

Bash Scripting

From your first terminal command to scripts that loop, branch, process text and handle signals.



17 lessons

R Programming

Data structures, cleaning, visualization, functions and a first tour of modeling.



17 lessons

Python Programming

The Python counterpart of the R course: arrays and DataFrames, cleaning, plots, functions and a first tour of modeling.

LaTeX

22 lessons

LaTeX

Typeset documents by writing, not clicking: structure, math, tables, figures and packages.



8 lessons

Mathematical Epidemiology Prep

SIR and SEIR models, R_0, vaccination, stochastic outbreaks and model fitting, in R and Python.

Resources

terminal

```
git clone https://github.com/seknewna/Introduction-to-Computing-and-LaTeX.git
cd Introduction-to-Computing-and-LaTeX
```

```
bash-scripting/scripts/*.sh
r-programming/chapters/*.Rmd
python/chapters/*.py
python/notebooks/*.ipynb
latex/index.html
mathepi/r/*.Rmd
mathepi/python/*.py
```

Every script and chapter below is also shown in full on this page — clone only if you want to run them locally.



Bash Scripting

18 lessons · Instructor: Dr. Lema Logamou Seknewna

18 chapters, from your first terminal command to trap signals and named pipes. Course content adapted from Dave Eddy's free "Complete Bash Scripting Course" (ysap.sh), MIT-licensed source code.

Chapter 00 · 1 of 18

Introduction

Every script starts with a shebang line telling the system which interpreter to use. This chapter sets expectations for the course and gets your environment ready before you touch a single line of real scripting.

▼ View script

○ ○ ○ bash-scripting/scripts/00-introduction.sh

```
#!/usr/bin/env bash
#
# Chapter 00 — Introduction
# Real commands referenced in this lesson of the course.

echo "This course lives entirely in the terminal — no separate app needed."
echo "Every later script in this repo starts with the same shebang line:"
echo '  #!/usr/bin/env bash'
```

Terminal & File Basics

Before scripting comes fluency in the terminal itself: moving around the filesystem, creating and removing files, revealing hidden dotfiles, searching inside files with grep, and paging through output that's too long for one screen.

▼ View script

○ ○ ○ bash-scripting/scripts/01-terminal-and-file-basics.sh

```
#!/usr/bin/env bash
#
# Chapter 01 — Terminal & File Basics
# Real commands from this lesson: cd, mkdir, touch, cat, echo, rm, pwd,
# cp, alias, man, ls -a, ls -l, grep -A/-B/-C/-i/-o/-v, less/more.

mkdir -p foo && cd foo

pwd                          # confirm where you are (CWD)
touch lesson-1.txt
echo hi > lesson-1.txt
cat lesson-1.txt
cp lesson-1.txt lesson-2.txt
rm lesson-1.txt lesson-2.txt

echo
echo "Hidden files start with a dot and are skipped by plain ls:"
touch .lesson.txt
ls                           # .lesson.txt is not listed
ls -a                        # now it shows up
ls -l .lesson.txt
rm .lesson.txt

echo
echo "grep context flags — build a small file to search:"
printf 'a\nb\nc\n' > file.txt
grep -A1 b file.txt          # 1 line of context After the match
grep -i B file.txt           # case-Insensitive
grep -o b file.txt           # Only print the matched text
grep -v b file.txt           # Invert — show non-matching lines
rm file.txt

echo
echo "man rm shows -i (confirm before delete) — worth aliasing:"
echo "  alias rm='rm -i'"

cd .. && rm -rf foo
```

Man Pages, Variables & Permissions

Learn to read a command's manual instead of guessing, tell the difference between programs and shell builtins, assign your first variables, and understand the read/write/execute permission bits that gate what a file can do.

▼ View script

○ ○ ○ bash-scripting/scripts/02-man-pages-variables-permissions.sh

```
#!/usr/bin/env bash
#
# Chapter 02 — Man Pages, Variables & Permissions
# Real commands: man, help, compgen -b, which, type, file, $PATH,
# built-in variables, chmod, stat.

echo "man covers real programs on disk; bash builtins have their own help:"
echo "  man cp        # a real program"
echo "  help history  # history is a shell builtin, not a program"
echo "  compgen -b    # lists every builtin"

echo
echo "which finds programs on \$PATH; it fails on builtins — type works on both:"
which ls
type history

echo
echo "file tells you what kind of file something is:"
file /usr/bin/ls 2>/dev/null || file /bin/ls

echo
echo "\$PATH is a colon-separated list — split it onto separate lines:"
echo "$PATH" | tr ':' '\n' | head -5

echo
echo "A few variables bash sets for you:"
echo "PWD=$PWD"
echo "USER=$USER"
echo "SHELL=$SHELL"

echo
echo "Capture a command's output into a variable:"
now=$(date)
echo "now=$now"

echo
echo "Permissions — chmod +x is what makes a script directly runnable:"
touch demo-script.sh
stat demo-script.sh | head -4
chmod +x demo-script.sh
ls -lha demo-script.sh
rm demo-script.sh
```

Finally Scripting

Your first real scripts: reading user input, packaging logic into functions, branching with if/else, repeating work with for loops, and redirecting input and output between commands and files.

▼ View script

○ ○ ○ bash-scripting/scripts/03-finally-scripting.sh

```
#!/usr/bin/env bash
#
# Chapter 03 — Finally Scripting
# Real commands: read -p, positional args ($1, $@), functions with
# local, if/[[ ]], for loops (list, range, C-style), I/O redirection.

echo "Take input from the user, or fall back to it if no argument was given:"
if [[ -n $1 ]]; then
    name=$1
else
    name='course-student'   # in place of an interactive read -p prompt
fi
echo "hello $name!"

echo
echo "A function using 'local' so it doesn't leak into the caller's scope:"
my-func() {
    local arg1=$1
    echo "hello from my function, arg1 is $arg1"
}
my-func "$@"

echo
echo "Return codes are separate from printed output:"
check-even() {
    (( $1 % 2 == 0 ))
}
check-even 4 && echo "4 is even (exit code: $?)"

echo
echo "The three for-loop shapes covered in this lesson:"
for name in foo bar baz; do echo "name is $name"; done
for c in {a..c}; do echo "c is $c"; done
for ((i = 0; i < 3; i++)); do echo "i is $i"; done

echo
echo "Reading lines piped in from another command:"
printf 'one\ntwo\n' | while read -r line; do
    echo "got line: $line"
done
```

Case Statements, Arrays & Substitution

Case statements offer a cleaner alternative to long if/elif chains. This chapter also introduces indexed and associative arrays, the IFS variable that controls word-splitting, command substitution, arithmetic expressions, and process substitution.

▼ View script

○ ○ ○ bash-scripting/scripts/04-case-arrays-substitution.sh

```
#!/usr/bin/env bash
#
# Chapter 04 — Case Statements, Arrays & Substitution
# Real commands: case/esac, indexed & associative arrays, IFS,
# $(...) command substitution, $((...)) arithmetic, <(...) process sub.

for name in dave bill anyone; do
    case "$name" in
        d*)  echo "matched d*: $name" ;;
        b*)  echo "matched b*: $name" ;;
        *)   echo "matched *: $name"  ;;
    esac
done

echo
echo "Indexed array — note *the difference* between [*] and [@]:"
array=(foo bar baz)
echo "element 0: ${array[0]}"
echo "last element: ${array[-1]}"
echo "all (as one word): ${array[*]}"
echo "all (as separate words): ${array[@]}"

echo
echo "Associative array — string keys instead of positions:"
declare -A capitals=([France]=Paris [Japan]=Tokyo)
for key in "${!capitals[@]}"; do
    echo "$key is ${capitals[$key]}"
done

echo
echo "Command substitution — prefer \$(...) over legacy backticks:"
thing=$(whoami)
echo "thing is $thing"

echo
echo "Arithmetic expression:"
i=0
((i++))
echo "i is $i"
echo "2 + 2 = $((2 + 2))"

echo
```

```
echo "Process substitution treats a command's output like a file:"
diff <(printf 'a\nb\n') <(printf 'a\nc\n') || true
```

Text Processing

The classic Unix text tools: cut for pulling out fields, tr for character translation, sed for stream editing, awk for column-based processing, grep for filtering by pattern, and find for locating files across a directory tree.

▼ View script

○ ○ ○ bash-scripting/scripts/05-text-processing.sh

```
#!/usr/bin/env bash
#
# Chapter 05 — Text Processing
# Real commands: cut, tr, sed, awk -F, grep -c, find -type/-name/-exec.

echo "name,age,city" > people.csv
echo "Alice,30,Boston" >> people.csv
echo "Bob,25,Denver" >> people.csv

echo "cut -d',' -f2 — pull just the age column:"
cut -d',' -f2 people.csv

echo
echo "sed — swap Boston for a placeholder:"
sed 's/Boston/[city]/' people.csv

echo
echo "awk -F',' — print field 1 and field 3, skipping the header:"
awk -F',' 'NR > 1 { print $1, $3 }' people.csv

echo
echo "grep -c — count matching lines instead of printing them:"
grep -c 'Alice' people.csv

rm people.csv

echo
echo "find — locate files by type and name, then act on each match:"
find . -maxdepth 1 -type f -name "*.sh"
find . -maxdepth 1 -type f -name "*.sh" -exec echo "found: {}" \;
```

Bash Arguments, Pipe Status & Timing

Read arguments passed into your script, inspect the exit status of every stage in a pipeline with PIPESTATUS (not just the last one), and measure how long a command takes to run.

▼ View script

○ ○ ○ bash-scripting/scripts/06-arguments-pipes-timing.sh

```
#!/usr/bin/env bash
#
# Chapter 06 — Bash Arguments, Pipe Status & Timing
# Real commands: bash -x (trace), bash -n (syntax check), bash -u
# (error on unset vars), PIPESTATUS, time.

cat > sample.sh << 'INNER_EOF'
#!/usr/bin/env bash
first_name='john'
last_name='eddy'
echo "Hello $first_name $last_name!"
INNER_EOF

echo "bash -n only checks syntax, it does not run the script:"
bash -n sample.sh && echo "syntax OK"

echo
echo "bash -x traces every command as it executes:"
bash -x sample.sh

rm sample.sh

echo
echo "PIPESTATUS holds the exit code of every stage in a pipeline, not"
echo "just the last one that \$? would give you:"
true | false | true
echo "PIPESTATUS: ${PIPESTATUS[@]}"

echo
echo "time reports real/user/sys time for a command:"
time sleep 0.2
```

Sourcing Code, Subshells & Return vs. Output

Sourcing loads another file's code into your current shell, while curly braces { } and parentheses ( ) control whether commands run in your current shell or an isolated subshell. Also covers the difference between a function's numeric return status and its printed output.

▼ View script

○ ○ ○ bash-scripting/scripts/07-sourcing-and-subshells.sh

```
#!/usr/bin/env bash
#
# Chapter 07 — Sourcing Code, Subshells & Return vs. Output
# Real commands: source, { } vs ( ), capturing stdout vs stderr
# separately when calling a function.

cat > lib.sh << 'INNER_EOF'
greet() {
    local name=$1
    echo "hello $name!"
}
INNER_EOF

source lib.sh      # loads greet() into THIS shell
greet dave
rm lib.sh

echo
echo "{ } runs in the current shell; ( ) runs in an isolated subshell:"
{ set_in_brace="yes"; }
echo "set_in_brace=$set_in_brace"
( set_in_paren="yes" )
echo "set_in_paren='${set_in_paren:-unset}'"

echo
echo "A function's stdout and stderr can be redirected independently:"
my-func() {
    echo "this goes to stdout" >&1
    echo "this goes to stderr" >&2
}
output=$(my-func 2>/dev/null)   # keep stdout, throw away stderr
echo "captured: $output"
```

Parameter & Array Expansion

Bash's parameter expansion syntax packs a lot of power into a small space: default values for unset variables, trimming prefixes and suffixes off strings, search-and-replace, and slicing arrays by index.

▼ View script

○ ○ ○ bash-scripting/scripts/08-parameter-and-array-expansion.sh

```
#!/usr/bin/env bash
#
# Chapter 08 — Parameter & Array Expansion
# Real commands: case conversion (^ , ,), default/required values
# (:-  ?), replacements (# ## % // ), substrings, array slicing.

s='dave eddy'
echo "uppercase first letter: ${s^}"
echo "uppercase everything:   ${s^^}"

echo
name=${1:-dave}
echo "default if unset: $name"

path=/home/dave/data.txt
echo "strip shortest prefix:  ${path#*/}"
echo "strip longest prefix:   ${path##*/}"
echo "strip shortest suffix:  ${path%/*}"
echo "replace first match:    ${path/a/o}"
echo "replace all matches:    ${path//a/o}"

echo
echo "Substrings by offset and length:"
echo "${s:0:4}"
echo "${s: -4}"

echo
echo "Loop a string one character at a time using substring expansion:"
len=${#s}
for ((i = 0; i < len; i++)); do
    printf '%s' "${s:i:1}"
done
echo

echo
echo "Array expansion works the same way, applied to every element:"
arr=(foo bar baz)
printf '<%s>\n' "${arr[@]/a/1}"
```

Globbing

Wildcards like * and ? let you match filenames without listing them one by one. This chapter adds extended globbing for more expressive patterns and shell options like nullglob that change how a non-matching glob behaves.

▼ View script

○ ○ ○ bash-scripting/scripts/09-globbing.sh

```
#!/usr/bin/env bash
#
# Chapter 09 — Globbing
# Real commands: *, ?, [..] character classes, extglob's !( ) +( ) @( ),
# and the nullglob/dotglob/globstar shell options.

mkdir -p files
touch files/foo.txt files/foo.jpg files/bar.txt files/baz.jpg

echo "Basic wildcard — every .txt file:"
printf '%s\n' files/*.txt

echo
echo "? matches exactly one character:"
printf '%s\n' files/ba?.txt

echo
echo "[..] matches one character from a set:"
printf '%s\n' files/ba[rz].txt

echo
echo "Extended globbing (needs extglob enabled) — everything except .txt:"
shopt -s extglob
printf '%s\n' files/!(*.txt)
shopt -u extglob

echo
echo "nullglob — a non-matching glob expands to nothing instead of the"
echo "literal pattern string:"
shopt -s nullglob
printf '<%s>\n' files/*.md
shopt -u nullglob

rm -rf files
```

Brace Expansion

Brace expansion generates text from comma-separated lists or numeric ranges before the shell even looks at the filesystem — a fast way to produce sequences of filenames or values.

▼ View script

○ ○ ○ bash-scripting/scripts/10-brace-expansion.sh

```
#!/usr/bin/env bash
#
# Chapter 10 — Brace Expansion
# Real commands: {a,b,c} comma lists, {1..N} numeric ranges, and
# combining braces with a shared filename prefix.

echo "Comma list:"
echo {hello,world}

echo
filename='my-file'
echo "Prefix + comma list, useful for generating related filenames:"
echo "$filename."{txt,jpg,mov}

echo
echo "Numeric range with a step:"
echo {1..20..5}

echo
echo "Combined with globbing — braces don't require the files to exist,"
echo "unlike a glob pattern:"
mkdir -p files
touch files/{foo,bar}.txt
printf '<%s>\n' files/{foo,bar}.txt
rm -rf files
```

printf, Date Formatting & Regular Expressions

printf gives you precise control over formatted output, the date command formats timestamps however you need, [[=~]] matches strings against regular expressions, and mapfile reads input straight into an array.

▼ View script

○ ○ ○ bash-scripting/scripts/11-printf-dates-regex.sh

```
#!/usr/bin/env bash
#
# Chapter 11 — printf, Date Formatting & Regular Expressions
# Real commands: printf format specifiers (%s %d %5s %-5s %05d),
# date +FORMAT, printf '%(FORMAT)T', regex with [[ =~ ]], mapfile.

echo "printf needs a format string — passing data directly is a bug:"
s='hello world'
printf '%s\n' "$s"

echo
echo "Width and padding:"
printf '%5s|%-5s|\n' hi hi
printf '%05d\n' 55

echo
echo "Date formatting, two ways:"
datefmt='%Y-%m-%d %H:%M:%S'
date "+$datefmt"
printf "%($datefmt)T\n" -1

echo
echo "Matching a pattern with a regex:"
email='user@example.com'
if [[ "$email" =~ ^[^@]+@[^@]+\.[a-z]+$ ]]; then
    echo "'$email' looks like an email"
fi

echo
echo "mapfile reads lines straight into an array:"
printf 'one\ntwo\nthree\n' > lines.txt
mapfile -t lines < lines.txt
echo "line 2: ${lines[1]}"
rm lines.txt
```

Test Operators

The classic [ ] test command, the bash-only [[ ]] with safer parsing, and the test command itself — plus the string and file-check flags (-z, -f, -d, and friends) you'll reach for constantly in conditionals.

▼ View script

○ ○ ○ bash-scripting/scripts/12-test-operators.sh

```
#!/usr/bin/env bash
#
# Chapter 12 — Test Operators
# Real commands: help '[[', help '[', help test, and quoted
# ANSI-C ($'...') strings.

touch demo.txt

[ -f demo.txt ]; echo "[ -f ] exit code: $?"
[[ -f demo.txt && -r demo.txt ]] && echo "[[ ]]: demo.txt exists and is readable"
test -f demo.txt && echo "test: demo.txt exists"

rm demo.txt

echo
echo "Plain echo doesn't interpret backslash escapes by default:"
echo 'line1\nline2'

echo
echo "\$'...' (ANSI-C quoting) does interpret them:"
echo $'line1\nline2'
```

Trap Signals & Named Pipes

trap intercepts signals like Ctrl+C so your script can clean up before exiting, and named pipes (FIFOs) let two separate processes pass data through a file-like path on disk.

▼ View script

○ ○ ○ bash-scripting/scripts/13-traps-and-named-pipes.sh

```
#!/usr/bin/env bash
#
# Chapter 13 — Trap Signals & Named Pipes
# Real commands: trap ... exit, mkfifo, reading/writing a FIFO path.

cleanup() {
    echo "cleanup function running"
}
trap cleanup exit

echo "script starting"
echo "..."
echo "script done"
# 'cleanup' fires here automatically because of the trap on exit

echo
echo "Named pipes let two processes talk through a path on disk:"
pipe="/tmp/course-pipe-$$"
mkfifo "$pipe"

echo "[client] hello" > "$pipe" &
read -r line < "$pipe"
echo "[reader] got: $line"

rm -f "$pipe"
```

Terminal Output & Color

ANSI escape codes add color and move the cursor around for progress indicators, and checking whether stdout is a real TTY keeps your script from dumping raw escape codes into a log file or pipe.

▼ View script

○ ○ ○ bash-scripting/scripts/14-terminal-output-and-color.sh

```
#!/usr/bin/env bash
#
# Chapter 14 — Terminal Output & Color
# Real commands: \e[...m ANSI escapes, tput setaf/sgr0, cursor
# save/restore with \e7 and \e8, and the [ -t 1 ] TTY check.

echo -e '\e[1mbold text\e[0m'
echo $'\e[31mred text\e[0m'

echo
echo "tput is a more portable way to reach the same codes:"
tput setaf 2; echo "green via tput"; tput sgr0

echo
echo "Save cursor position, print elsewhere, then restore it:"
printf '\e7'
printf '\e[5;10Hoverwritten text'
printf '\e8'
echo

echo
echo "Only use color if stdout is an actual terminal:"
if [[ -t 1 ]]; then
    echo "stdout is a TTY — color is safe here"
else
    echo "stdout is redirected — skip the escape codes"
fi
```

PS1 & Customizing Bash

Your shell prompt is just a variable (PS1) you can customize to show your user, host, and directory, and readline shortcuts like Ctrl+R for history search make everyday terminal use faster.

▼ View script

○ ○ ○ bash-scripting/scripts/15-prompt-and-customization.sh

```
#!/usr/bin/env bash
#
# Chapter 15 — PS1 & Customizing Bash
# Real commands: PS1 escape sequences, shopt -s cdspell/autocd,
# conditional aliases for ~/.bashrc.

echo "Current PS1:"
echo "$PS1"

echo
echo "A custom prompt using \u \h \w — user, host, working directory:"
echo '  PS1="\u@\h:\w\$ "'

echo
echo "Two interactive-only shell options worth turning on in .bashrc:"
echo '  shopt -s cdspell   # forgives small typos in cd targets'
echo '  shopt -s autocd    # typing a directory name alone cd'"'"'s into it'

echo
echo "A conditional alias — only defined if the feature is supported:"
echo '  ls --color=auto &>/dev/null && alias ls="ls --color=auto"'

echo
echo "Readline shortcuts worth practicing at the prompt (not scriptable):"
echo "  Ctrl+W   delete one word back"
echo "  Ctrl+U   delete to start of line"
echo "  Ctrl+R   reverse search through history"
```

Common Pitfalls

Field notes from real bugs: why parsing 'ls' output breaks on odd filenames, why aliases can't take arguments the way functions can, and why string length can surprise you with multibyte characters.

▼ View script

○ ○ ○ bash-scripting/scripts/16-common-pitfalls.sh

```
#!/usr/bin/env bash
#
# Chapter 16 — Common Pitfalls
# Real commands: the classic `for f in $(ls)` bug, alias argument
# limitations, and ANSI codes inflating string length.

mkdir -p files
touch files/a files/b "files/d      e"

echo "The fragile way — breaks on filenames containing spaces:"
for f in $(ls files); do
    echo "  bad: $f"
done

echo
echo "The safe way — glob directly, no ls, no word-splitting:"
for f in files/*; do
    echo "  good: $f"
done

rm -rf files

echo
echo "Aliases can't take positional arguments the way functions can:"
shopt -s expand_aliases
alias greet-alias='echo hello \$1'   # \$1 never gets replaced
greet-alias world

greet-func() { echo "hello $1"; }
greet-func world

echo
echo "ANSI color codes count toward string length even though they're"
echo "invisible on screen:"
colored=$'\e[33mHello\e[0m'
echo "visible text is 5 characters, but \${#colored} reports: ${#colored}"
```

Forkbombs & Wrap-up

A look at the infamous forkbomb one-liner as a cautionary piece of shell trivia — explained, never executed — followed by a recap of everything the course covered.

▼ View script

○ ○ ○ bash-scripting/scripts/17-forkbombs-and-wrapup.sh

```
#!/usr/bin/env bash
#
# Chapter 17 — Forkbombs & Course Wrap-up
#
# This chapter covers the classic bash forkbomb: a function that
# recursively spawns two copies of itself in the background, forever,
# exhausting process slots or memory. It's a well-known piece of shell
# trivia (search "bash fork bomb" for the one-liner), included here
# for recognition only — this file deliberately does not include or
# execute one.
#
# If you ever want to see the effect safely, do it only in a disposable
# VM or container, and cap your process count first:
#   ulimit -u 100

echo "A forkbomb defines a function that calls itself twice, piped"
echo "together and backgrounded, doubling the process count on every"
echo "generation until the system runs out of room."

echo
echo "That wraps up Part 1 — Bash Scripting."
echo "Next up: Part 2 — R Programming."
```



R Programming

17 lessons · Instructor: Dr. Lema Logamou Seknewna

17 chapters, from installing R to a first tour of PCA, clustering, and regression. Course content adapted from an introductory R course by Dr. Lema Logamou Seknewna (AIMS / National Institute of Statistics of Rwanda).

Chapter 00 · 1 of 17

Getting Started

Installing R and RStudio, setting up R Markdown so it can render to HTML, Word, and PDF, and getting the handful of extra tools (like tinytex) that later chapters depend on.

What is R?

R is a statistical programming language created in 1992 by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand, and first released as version 1.0 in February 2000. Its name comes from the initials of its creators' first names. R descends from the S language, which was developed at Bell Laboratories in the mid-1970s. See the official R project page for more history.

Install R

Download the build for your operating system from CRAN:

- Windows
- macOS
- Linux (Debian, Fedora/Red Hat, Ubuntu)

Install RStudio

RStudio (now Posit) is the IDE most R users work in day to day. Download it from posit.co/download/rstudio-desktop, or use Posit Cloud to run RStudio entirely in a browser with no install at all.

Setting up R Markdown

Required packages

- **tinytex** — a lightweight LaTeX distribution needed to render R Markdown documents

to PDF.

```
if (!"tinytex" %in% rownames(installed.packages())) {
  install.packages("tinytex")
}
library(tinytex)
install_tinytex() # installs LaTeX itself
```

- **Rtools** (Windows only) — needed to build packages from source. Download from cran.r-project.org/bin/windows/Rtools.

Once these are installed, R Markdown (`.Rmd`) files like the ones in this course can render to HTML, Word, or PDF directly from RStudio's Knit button.

▶ View R Markdown

Operators

Arithmetic operators, logical operators, and value-matching with match() and %in% — the small building blocks that every later script leans on.

R can act as a calculator directly from the console — every operator below can be typed and evaluated on the spot.

Arithmetic Operators

Addition: `+`

```
1 + 1
```

Subtraction: `-`

```
1 - 1
```

Multiplication: `*`

```
1 * 1
```

Division: `/`

```
1 / 1
```

Modulus (remainder of a division): `%%`

```
1 %% 2
```

Exponent: `^` or `**`

```
2 ^ 10 # or 2 ** 10
```

Integer division: `%/%`

```
1035 %/% 3
```

Logical Operators

Less than: `<`

```
1 < 1
```

Less than or equal to: `<=`

```
1 <= 1
```

Greater than: `>`

```
1 > 1
```

Greater than or equal to: `>=`

```
1 >= 1
```

Exactly equal to: `==`

```
"R" == "r"
```

R is case-sensitive — `"R"` and `"r"` are different values.

The equality operator can also compare one element against several at once:

```
"Species" == c("Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width", "Species")
```

Not equal to: `!=`

```
1 != 1
```

Negation / NOT: `!`

Flips `TRUE` to `FALSE` and vice versa.

```
!TRUE  # or !T
!FALSE # or !F

!(TRUE & FALSE) # this is TRUE
!(FALSE | TRUE) # this is FALSE
```

AND: `&`

```
TRUE & TRUE
TRUE & FALSE
FALSE & FALSE
```

OR: `|`

```
TRUE | TRUE
TRUE | FALSE
FALSE | FALSE
```

Value Matching

`match()` returns the position of the first match of its first argument in its second argument. Check `?match` for the full documentation.

```
match("Species", c("Sepal.Length", "Sepal.Width", "Petal.Length",
                   "Petal.Width", "Species"))
```

`%in%` checks whether a value exists anywhere in a vector, returning `TRUE`/`FALSE` instead of a position:

```
"Species" %in% c("Sepal.Length", "Sepal.Width", "Petal.Length",
                 "Petal.Width", "Species")
```

▶ **View R Markdown**

R Objects, Assignment & Data Types

Assigning values with <-, =, and ->, the rules for valid variable names, R's core data types (numeric, integer, complex, character, logical, raw, factor), and how to convert cleanly between them.

Assignment

R accepts three assignment operators: `<-`, `=`, and `->`.

A variable name:

- can begin with a letter or a dot. E.g. `a <- 1`, `0 -> .a`
- cannot contain spaces — use `_` or `.` instead
- can contain numbers, just not as the first character. E.g. `a1 <- 1`

```
v rsion <- 4.3.2 # invalid: a space inside the name breaks the parser
```

```
a <- 1
b <- 2
0 -> .a
a1 <- .a
```

Data Types

R has these core data types: numeric, integer, complex, character, logical, raw, and factor.

Numeric / double

Examples: `10.5`, `55`, `787`, `pi`.

```
PI <- pi; class(PI); typeof(PI)
n <- 55; class(n); typeof(n)
```

Integer

Append `L` to a literal to declare it as an integer (`1L`, `55L`, `100L`).

```
n <- 55L
class(n)
```

Complex

An example of a complex number is $9 + 3i$, where i is the imaginary part. Multiplying a real number by `1i` converts it to complex.

```
z <- 9 + 3i
class(z)
typeof(z)

z1 <- a + 1i * b
print(z1)
class(z1)
```

Character / string

```
string <- "I am Learning R"
class(string)
```

Remember: `"LeaRning"` is different from `"Learning"` — R is case-sensitive.

Logical / Boolean

```
TRUE  # or T
FALSE # or F
```

A logical value is often the *result* of a test:

```
"LeaRning" == "Learning"
```

Raw

```
text <- "I am learning R."
(raw_text <- charToRaw(text))
class(raw_text)
```

Converting raw back to text:

```
rawToChar(raw_text)
```

Factors

Factors represent qualitative/categorical data — colors, pass/fail, movie ratings — and are the backbone of statistical modeling in R.

```
Gender <- factor(c("Female", "Male"))
print(Gender)
class(Gender)
```

Logical, revisited

```
v <- TRUE
w <- FALSE

class(v); typeof(v)
!v

isTRUE(w)
```

Creating objects directly

Numeric:

```
n <- 10
x <- numeric(n) # a numeric vector of length n, filled with zeros
print(x)

x[1] <- 2.5
print(x)
```

Integer — note that assigning a decimal silently upgrades it to numeric:

```
n <- 10
x <- integer(n)
print(x)
class(x)

x[1] <- 2.5 # R converts the whole vector to numeric
class(x)
print(x)
```

Conversions

To convert between data types, use one of the `as.*()` functions from base R. The naming convention is `as.` followed by the target type.

- Numeric to character

```
n <- 55
n_char <- as.character(n)
print(n_char); class(n_char)
```

- Character to numeric

```
s <- "123"
s_num <- as.numeric(s)
print(s_num); class(s_num)
```

A string that isn't a valid number becomes `NA` (with a warning) rather than causing an error:

```
suppressWarnings(as.numeric("abc"))
```

- Factor to character

```
f <- factor(c("low", "high", "medium"))
f_char <- as.character(f)
print(f_char); class(f_char)
```

- Character to factor

```
level <- "medium"
level_fct <- as.factor(level)
print(level_fct); class(level_fct)
```

▶ View R Markdown

Data Structures I: Vectors & Matrices

Building vectors with seq(), rep(), and sample(), then moving into matrix algebra — transpose, inverse, solving linear systems, and eigenvalues — capped off with a hand-rolled PCA on the iris dataset.

R's most-used data structures are vectors, matrices, arrays, lists, factors, and data frames. This chapter covers the first two.

Scalars and Vectors (1D)

A scalar is any single number; a vector is a collection of values of the same type (and can represent a sequence).

Mixing types inside `c()` forces R to coerce everything to one common type:

```
v <- c(1, "R", TRUE, FALSE, NA)
print(v)
class(v) # everything becomes character (NA is compatible with any type)
```

```
v2 <- c(1, 4, 8, FALSE, TRUE, FALSE, FALSE, TRUE, "R" == "r")
print(v2)
class(v2) # everything becomes numeric: FALSE -> 0, TRUE -> 1
```

Sequences with `seq()`

```
x <- seq(0, 2 * pi, length.out = 90)
length(x)
unique(round(diff(x), 5)) # the common step size

head(x, 5); tail(x)
range(x)
```

```
diff(c(4, 6))
diff(range(x)) # same as max(x) - min(x)
```

Indexing with a non-integer floors to the nearest integer:

```
x[11.8] == x[11]
as.integer(9.123486785) # what R does internally with a fractional index
```

The length of a vector:

```
length(x)
a <- 9
length(a) # a scalar is just a vector of length 1
```

Repetition with `rep()`

```
rep("I learn R", 3)
rep(c(0, 1), 5)
rep(c(0:1), c(3, 3))
rep(c(0:1), each = 3)
```

Random sampling

```
set.seed(123) # fixes the randomness for reproducibility
y <- sample(0:1, size = 200, replace = TRUE, prob = c(1/3, 1 - 1/3))
sum(y == 0)   # count of zeros
sum(y == 1)   # count of ones
```

`table()` counts occurrences for you:

```
table(y)
```

`prop.table()` converts counts to proportions:

```
prop.table(table(y))
mean(y == 0) # a shortcut: mean() of a logical vector is a proportion
```

A small survey example

```
Gender <- c("Female", "Male", "Male", "Male", "Female", "Female", "Male",
"Male", "Female", "Female", "Male", "Male", "Male", "Male", "Female",
"Female", "Male", "Female", "Female", "Male", "Female", "Male",
"Male", "Male", "Male", "Female", "Femaale", "Male", "Female",
"Male", "Male", "Female", "Male", "Female", "Malle", "Male", "Female",
"Female", "Femmale", "Female", "Female", "Female", "Female", "Male",
"Male", "Male", "Male")

sum(Gender == "Female")
mean(Gender == "Female")
table(Gender) # note the 3 misspelled entries
```

Cleaning up the typos:

```
Gender[Gender == "Femaale"] <- "Female"
Gender[Gender == "Femmale"] <- "Female"
Gender[Gender == "Malle"]   <- "Male"
table(Gender)
```

Sub-setting with `sample()`:

```
set.seed(76)
table(sample(Gender, size = 20))
```

Vector arithmetic

```
k <- 2
v <- c(1, 0, 3)

k + v; v + k
k - v; v - k
k * v; v * k
v / k
k / v
```

```
k <- 1/2
v <- c(pi, 0, 1, 4)
w <- c(0, pi, pi, 0, 0)

k * v; v * k
v ^ k
v + w
```

Matrices (2D)

```
(A <- matrix(1:25, ncol = 5, nrow = 5)) # column-major fill by default
(B <- matrix(1:25, nrow = 5, ncol = 5, byrow = TRUE))
```

Building a matrix by hand

```
v <- c(1, rep(0, 2), 1, 0, rep(1, 3), rep(0, 2), rep(1, 2))
A <- matrix(v, 3, byrow = TRUE)
print(A)
dim(A)
```

The same result using `dim<-`:

```
v <- c(1, rep(0, 2), 1, 0, rep(1, 3), rep(0, 2), rep(1, 2))
dim(v) <- c(4, 3)
v <- t(v)
v
```

A zero matrix:

```
n <- 6; p <- 8
matrix(0, nrow = n, ncol = p)
```

```
(A <- matrix(c(1, 0, 2, 5, 2, 1, 4, 2, 0), nrow = 3))
(B <- matrix(c(2, 5, 2, 3, 1, 1, 0, 1, 1), nrow = 3))
```

Matrices from vectors: `cbind` / `rbind`

```
v1 <- c(1, 0, 2); v2 <- c(5, 2, 1); v3 <- c(4, 2, 0)
(M1 <- cbind(v1, v2, v3))
(M2 <- rbind(v1, v2, v3))
```

Matrix operations

Transpose:

```
(A_T <- t(A))
```

Addition / subtraction:

```
A + B
A - B
```

Multiplication (`%*%`, not `*`):

```
ncol(A) == nrow(B)
A %*% B
```

Inverse:

```
(A_inv <- solve(A))
round(A %*% A_inv) # sanity check: should be the identity matrix
```

Solving a system of equations

$$\begin{cases} 2x + 2y &= 0 \\ x + 3y &= 2 \end{cases}$$

```
A1 <- matrix(c(2, 2, 1, 3), nrow = 2, byrow = TRUE)
b <- c(0, 2)
solve(A1, b)
```

Eigenvalues / eigenvectors

Eigenvalues λ of a square matrix A solve the characteristic equation $\det(A - \lambda I_n) = 0$. They're the mathematical basis of Principal Component Analysis, covered in Chapter 15.

```
ev <- eigen(A)
print(ev$values)

P <- ev$vectors
round(solve(P) %*% A %*% P, 4) # recovers a diagonal matrix of eigenvalues
```

Worked exercise: manual PCA on `iris`

1. Remove the last column (`Species`) from `iris`:

```
data(iris)
dta <- iris[, -5]
```

2. Compute the average of each remaining column:

```
avg <- colMeans(dta)
print(avg)
```

3. Center the data by subtracting each column's average:

```
scaled_dta <- scale(dta, center = TRUE, scale = FALSE)
head(scaled_dta, 3)
```

4. Eigen-decompose `t(scaled_dta) %*% scaled_dta`:

```
cov_mat <- t(scaled_dta) %*% scaled_dta
ev <- eigen(cov_mat)
ev$values
```

5. Project onto the first two eigenvectors:

```
P <- ev$vectors[, 1:2]
projection <- scaled_dta %*% P
head(projection)
```

This is exactly what `prcomp()` does under the hood:

```
pca <- prcomp(dta, center = TRUE, scale. = FALSE)
head(pca$x[, 1:2])
```

▶ View R Markdown

Data Structures II: Arrays, Lists & Data Frames

Multi-dimensional arrays, lists that hold mixed types together, and data frames — including turning a continuous age variable into readable age-group categories.

Arrays

Arrays generalize matrices to more than two dimensions.

```
(aRray <- array(1:24, dim = c(3, 4, 2)))
class(aRray)
```

A classic use case is gridded (NetCDF-style) data: longitude as columns, latitude as rows, and time as the third dimension — each time slice being its own row x column matrix.

```
dim(aRray)
aRray[1, 1, 2] # row 1, column 1, of the 2nd matrix "slice"
```

Lists

A list holds objects of different types and sizes together.

```
L <- list()
A <- matrix(0, 5, 6)

L$A <- A
L$my_seq <- sample(10, 5)
L$Bool <- TRUE
L[["iris_head"]] <- head(iris, 3)
length(L)

L[[5]] <- "I am learning R"
names(L)
names(L)[5] <- "String"
names(L)
```

```
mylist <- list("matrix" = A,
               "sequence" = 1:5,
               "Bool" = TRUE)
mylist$matrix
class(mylist[[1]])
mylist[c("matrix", "Bool")]
```

Data Frames

A data frame is a table of n rows (observations) by p columns (variables), where each column can hold a different data type.

Converting a continuous variable into categories

```
set.seed(123)
n <- 120
df <- data.frame(
  "ID"  = paste0("Particip_", 1:n),
  "age" = sample(0:120, size = n)
)
head(df, 5)
```

```
brks <- seq(0, 120, by = 10)
df$age_groups <- cut(df$age, breaks = brks, include.lowest = TRUE)
head(df, 5)
```

Exercise: summarizing the age groups

1. Counts by age group:

```
(counts <- table(df$age_groups))
```

2. A barplot of the age groups:

```
x <- barplot(counts, col = "purple", main = "Distribution of ages", xaxt = "n")
text(x = x, y = -1.5, cex = 0.8, xpd = TRUE, srt = 45, labels = names(counts))
```

3. A histogram of raw ages:

```
hist(df$age, main = "Histogram of ages", xlab = "age", col = "pink", breaks = 12)
```

4. IDs of participants in the (70, 80] age group:

```
df$ID[df$age_groups == "(70,80]"]
```

```
l <- split(df, df$age_groups)
names(l)
l$`(70,80]`$ID
```

Creating a data frame directly

```
df2 <- data.frame(x = rnorm(10), y = rpois(10, 2))
head(df2, 3)
```

▶ View R Markdown

Data Cleaning: Missing Values & Outliers

Finding and filling NAs, recovering numbers trapped in messy strings, and flagging outliers with the IQR method before imputing them away.

Missing Values (`NA`)

```
x <- c(NA, 1, 2, NA, 3, NA, 3.55)
which(is.na(x))  # which positions are missing
which(x >= 2)    # which positions are >= 2 (NAs drop out silently)
which(!is.na(x)) # which positions are NOT missing
```

Replacing missing values with the mean of the non-missing ones:

```
x[is.na(x)] <- mean(x, na.rm = TRUE)
print(x)
```

NAs introduced by coercion

Converting a messy character vector straight to numeric turns anything non-numeric into `NA`:

```
x <- c("2", "1", "2", "7", "$8", "3", "2.5", "9", "2,7")
y <- suppressWarnings(as.numeric(x))
print(y)

id <- which(is.na(y))
id
```

Cleaning the offending values with `gsub()` before converting:

```
x <- c("2", "1", "2", ",7", "$8", "3", "&2.5", "9", "2,7")
x <- gsub("\\$|\\&", "", x)   # drop currency symbols
x <- gsub(",", ".", x)        # normalize decimal commas to dots
y <- as.numeric(x)
print(y)
```

Outlier Detection

The Inter-Quartile Range (IQR) method

A value outside the interval below is treated as an outlier:

$$[Q_1 - 1.5 \times IQR,\ Q_3 + 1.5 \times IQR]$$

where Q_1/Q_3 are the 25th/75th percentiles and $IQR = Q_3 - Q_1$.

```
x <- c(19.103, 19.632, 22.494, 20.113, 10.5, 22.744, 20.737, 17.976,
       18.901, 192.87, 21.959, 0.001, 20.641, 20.177, 19.111, 22.859,
       207.97, 16.853, 21.122, 19.244, 18.291, 19.651, 18.358, 18.834, 19)

x_copy <- x
boxplot(x_copy, main = "Before handling outliers")
```

```
Q <- as.numeric(quantile(x, probs = c(0.25, 0.5, 0.75)))
d <- as.data.frame(t(Q))
names(d) <- paste0("Q", 1:length(Q))
d$IQR      <- d$Q3 - d$Q1
d$LowerLim <- d$Q1 - 1.5 * d$IQR
d$UpperLim <- d$Q3 + 1.5 * d$IQR
d
```

Flagging every value as an outlier or not:

```
(d_out <- data.frame(x = x, outlier = x < d$LowerLim | x > d$UpperLim))
```

Two equivalent ways to pull out just the outliers:

```
x[d_out$outlier]                                  # using our own table
(bxpt <- boxplot(x, range = 1.5, plot = FALSE))$out # using boxplot()'s own $out
```

Imputing the outliers

Treat outliers as missing, then fill them with the mean of everything else:

```
x[d_out$outlier] <- NA
x[is.na(x)] <- mean(x, na.rm = TRUE)
```

```
par(mfrow = c(1, 2))
boxplot(x_copy, main = "Before\nhandling outliers", cex.main = 0.8)
boxplot(x, main = "After\nhandling outliers", col = "green", cex.main = 0.8)
```

No more outliers remain after imputation.

▶ View R Markdown

Charts & Distribution Simulations

Base R plotting — bar, pie, histogram, scatter — then simulating uniform, binomial, Gaussian, exponential, and Poisson distributions, plus fitting and interpreting a simple linear model.

Base R Charts

Bar chart

```
frequencies <- c(Female = 35, Children = 25, Male = 40)
barplot(frequencies, cex.names = 0.8,
        names.arg = names(frequencies),
        col = c("purple", "salmon", "pink"))
```

`cex.names` shrinks the axis labels; lower it further if labels get cut off.

Pie chart

```
par(mfrow = c(1, 2))
pie(frequencies, cex = 0.8,
    labels = paste0(names(frequencies), " (", frequencies, ")"),
    col = c("purple", "salmon", "pink"))
pie(frequencies, cex = 0.8,
    labels = paste0(names(frequencies), " (", frequencies, ")"),
    col = rainbow(length(frequencies)))
```

Histograms and density

```
set.seed(12092024)
x <- sample(1:120, size = 1000, replace = TRUE)
par(mfrow = c(1, 2))
hist(x, probability = TRUE) # densities instead of raw counts
plot(density(x), col = "salmon", main = "density of x")
```

```
with(iris, hist(Sepal.Length, probability = TRUE))
lines(density(iris$Sepal.Length), col = "red")
```

Scatter plot

```
x <- seq(0, 2 * pi, length.out = 50)
y <- sin(x)
plot(x, y, pch = 16, col = "blue")
```

Exercise: scatter plot of `Petal.Length` vs `Petal.Width`, colored by species, with a fitted regression line.

```
f <- Petal.Length ~ Petal.Width
lr <- lm(f, data = iris)
plot(f, data = iris, col = iris$Species, pch = 16)
abline(lr, col = "blue", lwd = 2, lty = 3)
legend("topleft", legend = unique(iris$Species),
       col = unique(iris$Species), pch = 16, bty = "n")
```

Distribution Simulations

Uniform distribution

$$f_X(x) = \begin{cases} \frac{1}{b-a} & \text{if } x \in [a, b] \\ 0 & \text{else} \end{cases}$$

```
set.seed(123)
x <- runif(1000)
hist(x, probability = TRUE, xlab = NULL,
     main = "Histogram of a uniform distribution", col = "turquoise")
lines(density(x), col = "red", lwd = 2, lty = 3)
abline(v = mean(x), col = "blue", lty = 2, lwd = 2)
legend("topleft", lty = c(2, 2), col = c("red", "blue"),
       legend = c("Density", "Average"))
```

Testing whether a sample looks uniform vs. normal:

```
ks.test(x, "punif") # p > 0.05 supports "looks uniform"
qqnorm(x)
qqline(x, col = "red")
```

Binomial distribution

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

```
rbinom_dist <- rbinom(n = 10000, size = 1, prob = 0.3)
par(mfrow = c(1, 2))
pie(table(rbinom_dist))
barplot(table(rbinom_dist))
```

A histogram isn't well suited to discrete data like this — bar/pie charts work better.

Gaussian (normal) distribution

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left\{-\frac{1}{2\sigma^2}(x-\mu)^2\right\}$$

```
set.seed(13092024)
gauss_dist <- rnorm(1000, mean = 0, sd = 1)
hist(gauss_dist, probability = TRUE, breaks = 30, xlab = NULL,
     main = "Histogram of a standard normal distribution", col = "turquoise")
lines(density(gauss_dist), col = "red", lwd = 2, lty = 2)
abline(v = mean(gauss_dist), col = "blue", lty = 2, lwd = 2)
```

```
mean(gauss_dist)
sd(gauss_dist)
```

Simulating a linear relationship

$$y_i = \alpha + \beta x_i + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, 1)$$

```
set.seed(123)
x <- rnorm(1000, mean = 5, sd = 1.5)
error_term <- rnorm(1000)
z <- 4 + 8 * x + error_term # true relationship, known only because we simulated it

df <- data.frame(x, z)
head(df, 5)
```

Fitting a simple linear model:

```
LR <- lm(z ~ x, data = df)
coefs <- coefficients(LR)
summary(LR)

par(mfrow = c(2, 2))
plot(LR) # standard diagnostic plots
```

Estimated model: $\hat{y} = 3.75 + 8.06\,x$. Predicting for a new value, $x = 15$:

```
new_data <- data.frame(x = 15)
(new_data$predicted <- predict(object = LR, newdata = new_data))
```

```
plot(predicted ~ x, data = new_data, col = "darkgreen", xlim = c(0, 15),
     ylim = c(0, new_data$predicted), pch = 16, ylab = "z")
points(z ~ x, data = df, col = "darkorange", pch = 20)
abline(LR, col = "blue")
legend("topleft", c("Actual", "Predicted"),
       col = c("darkorange", "darkgreen"), pch = c(20, 16), bty = "n")
```

Exponential distribution

```
set.seed(13092024)
exp_dist <- rexp(1000, rate = 1)
hist(exp_dist, probability = TRUE, xlab = NULL,
     main = "Histogram of exponential distribution", col = "turquoise")
lines(density(exp_dist), col = "red", lwd = 2, lty = 2)
abline(v = mean(exp_dist), col = "blue", lty = 2, lwd = 2)
```

Poisson distribution

```
set.seed(13092024)
pois_dist <- rpois(1000, lambda = 2.5)
hist(pois_dist, probability = TRUE, main = "Histogram of Poisson distribution",
     col = "turquoise", xlab = NULL)
lines(density(pois_dist), col = "red", lwd = 2, lty = 2)
abline(v = mean(pois_dist), col = "blue", lty = 2, lwd = 2)
```

▶ View R Markdown

Flow Control: Conditionals & Loops

if/else, for, while, and repeat, worked through a student-grading system, a dice-streak game, and looping over a folder of files to import them all at once.

`if` / `else`

```
if (condition) {
  ## code to run
}
```

```
x <- 3
if (x < 4) {
  print(TRUE)
} else {
  print(FALSE)
}

# one-liner form
if (x < 4) print(TRUE) else print(FALSE)

# vectorized form: ifelse() works element-by-element on a whole vector
x <- c(3, 4, 6, 7, 0, 4)
ifelse(x < 4, TRUE, FALSE)
```

Nested conditions: grading example

```
set.seed(123)
marks <- round(rnorm(50, mean = 60, sd = 15), 2)
```

Categorize each mark: Fail [0-60), Pass [60-70), Good Pass [70-80), Very Good Pass [80-85), Distinction [85+].

Approach 1 — iterate over values directly:

```
category <- character(0)
for (m in marks) {
  if (m >= 0 & m < 60) {
    category <- c(category, "Fail")
  } else if (m >= 60 & m < 70) {
    category <- c(category, "Pass")
  } else if (m >= 70 & m < 80) {
    category <- c(category, "Good Pass")
  } else if (m >= 80 & m < 85) {
    category <- c(category, "Very Good Pass")
  } else {
    category <- c(category, "Distinction")
  }
}
head(df_marks <- data.frame(marks, category), 5)
```

Approach 2 — iterate over indices:

```
category2 <- character()
for (i in seq_along(marks)) {
  if (marks[i] >= 0 & marks[i] < 60) {
    category2[i] <- "F"
  } else if (marks[i] >= 60 & marks[i] < 70) {
    category2[i] <- "P"
  } else if (marks[i] >= 70 & marks[i] < 80) {
    category2[i] <- "GP"
  } else if (marks[i] >= 80 & marks[i] < 85) {
    category2[i] <- "VGP"
  } else {
    category2[i] <- "D"
  }
}
df_marks$category2 <- category2
head(df_marks, 5)
```

Approach 3 — vectorized with nested `ifelse()`:

```
category3 <- ifelse(marks < 60, "Fail",
              ifelse(marks < 70, "Pass",
               ifelse(marks < 80, "Good Pass",
                ifelse(marks < 85, "Very Good Pass", "Distinction"))))
all(df_marks$category == category3)
```

`for` Loops

```
m <- 10
for (i in 1:m) print(i)
```

```
for (i in 1:10) {
  if (i %% 3 == 0) {
    cat(i, "is a multiple of 3\n")
  } else if (i %% 2 == 0) {
    cat(i, "is a multiple of 2\n")
  } else {
    cat(i, "is neither a multiple of 2 nor 3\n")
  }
}
```

Exercise 1: odd or even

```
for (i in 1:10) {
  if (i %% 2 == 0) {
    cat(i, "is even\n")
  } else {
    cat(i, "is odd\n")
  }
}
```

Exercise 2: importing every CSV in a folder

Build the file path from parts using `paste()` / `paste0()`:

```
paste("data", "csv", sep = "/")
paste0("data", "/", "csv")
```

List every CSV in a folder, then load them all into one list:

```
dir.create("demo_csv", showWarnings = FALSE)
write.csv(head(iris, 3), "demo_csv/iris_part.csv", row.names = FALSE)
write.csv(head(mtcars, 3), "demo_csv/mtcars_part.csv", row.names = FALSE)

file_names <- dir("demo_csv", pattern = "\\.csv$", full.names = TRUE)
file_names

data_list <- list()
for (f in file_names) {
  data_list[[basename(f)]] <- read.csv(f)
}
str(data_list)
unlink("demo_csv", recursive = TRUE) # clean up the demo files
```

`while` Loops

```
while (condition) {
  ## code to run
  ## something that eventually makes condition FALSE
}
```

```
i <- 0
while (i <= 10) {
  print(i * 2)
  i <- i + 10
}
```

Exercise: dice streak game

Print "YOU WON!" the moment three 5s are rolled in a row.

```
set.seed(42)
streak <- 0
rolls <- 0
repeat {
  rolls <- rolls + 1
  roll <- sample(1:6, 1)
  streak <- if (roll == 5) streak + 1 else 0
  if (streak == 3) {
    cat("YOU WON! (after", rolls, "rolls)\n")
    break
  }
}
```

Exercise: interactive multiplication table

This one needs a live console, since `readline()` only works in an interactive R session (not when a script is run non-interactively):

```
repeat {
  number <- as.numeric(readline(prompt = "Enter a number 0-9: "))
  cat(number, "x 2 table:\n")
  for (i in 1:10) cat(number, "x", i, "=", number * i, "\n")

  again <- readline(prompt = "Continue? (y/n): ")
  if (tolower(again) != "y") break
}
```

`repeat`

```
i <- 0
repeat {
  ## code to run
  i <- i + 1
  if (something_happens) break
}
```

```
i <- 0
repeat {
  print(i)
  i <- i + 1
  if (i > 5) break
}
```

▶ **View R Markdown**

The Apply Family of Functions

apply, sapply, lapply, tapply, and vapply as faster, more readable alternatives to writing an explicit loop every time.

The `apply` family replaces explicit loops when you want to run the same function across rows, columns, or elements of a data structure.

`apply()`

Applies a function across the margins (rows or columns) of a matrix.

```
A <- matrix(1:4, nrow = 2)
A

avg <- function(x) sum(x) / length(x)
avg(1:10)
```

```
apply(iris[-5], MARGIN = 2, summary) # MARGIN = 2 means "column-wise"
```

`sapply()`

Like `apply()`, but works over vectors/lists and simplifies the result to a vector or matrix where possible.

```
sapply(A, sum) # does not respect matrix structure — flattens first
```

```
coef_disp <- function(x) sd(x) / mean(x)
get_range  <- function(x) diff(range(x))

sapply(Filter(is.numeric, iris), get_range)

sapply(Filter(is.numeric, iris), function(x) c(
  min = min(x), max = max(x), std_dev = sd(x),
  Q1 = quantile(x, 0.25), Med = quantile(x, 0.5), Q3 = quantile(x, 0.75),
  disp = coef_disp(x)
))
```

`sapply()` falls back to returning a list when results aren't the same length:

```
sapply(iris, summary)
```

```
sapply(1:10, function(x) x^2)
```

`lapply()`

Always returns a list, one element per input element:

```
a <- lapply(iris[-5], mean)
unlist(a)
```

`tapply()`

Applies a function to subgroups of a vector, split by a factor:

```
tapply(iris$Sepal.Length, iris$Species, mean)
```

`vapply()`

Like `sapply()`, but requires you to specify the expected output shape — safer for use inside functions/packages since it fails loudly on a mismatch:

```
vapply(as.list(iris[-5]), quantile,
       FUN.VALUE = c("0%" = 0, "25%" = 0, "50%" = 0, "75%" = 0, "100%" = 0))
```

▶ **View R Markdown**

Chapter 09 · 10 of 17

Writing Functions in R

Defining your own functions from scratch, then applying that to two worked exercises: replacing missing values with a chosen statistic, and solving a quadratic equation based on its discriminant.

Function Syntax

```
function_name <- function(arg1, arg2, ...) {
  ## code to run
}
```

```
toss_coin <- function() {
  sample(6, size = 1)
}

set.seed(1)
faces <- sapply(1:10, function(i) toss_coin())
faces
```

```
pp <- function(x) return(x + 1)
i <- 1
(i <- pp(i))
```

Exercises

1. Replace missing values with a summary statistic

```
replace_missing <- function(x, fun = mean) {
  x[is.na(x)] <- fun(x, na.rm = TRUE)
  x
}

test_vec <- c(1, 2, NA, 4, NA, 6)
replace_missing(test_vec)
replace_missing(test_vec, fun = median)
```

2. Solve a quadratic equation and comment on its discriminant

For $ax^2 + bx + c = 0$:

```
                  ┌─ discriminant > 0 → two distinct real roots
compute disc = b² - 4ac ─┼─ discriminant = 0 → one repeated real root
                  └─ discriminant < 0 → two complex conjugate roots
```

```
solve_quadratic <- function(a, b, c) {
  if (a == 0) stop("a must be non-zero for a quadratic equation")

  disc <- b^2 - 4 * a * c

  if (disc > 0) {
    message("Two distinct real roots")
    roots <- c((-b + sqrt(disc)) / (2 * a), (-b - sqrt(disc)) / (2 * a))
  } else if (disc == 0) {
    message("One repeated real root")
    roots <- -b / (2 * a)
  } else {
    message("Two complex conjugate roots")
    real_part <- -b / (2 * a)
    imag_part <- sqrt(-disc) / (2 * a)
    roots <- c(complex(real = real_part, imaginary = imag_part),
               complex(real = real_part, imaginary = -imag_part))
  }
  roots
}

solve_quadratic(1, -3, 2)  # disc > 0
solve_quadratic(1, 2, 1)   # disc == 0
solve_quadratic(1, 0, 1)   # disc < 0
```

▶ **View R Markdown**

Packages

Installing packages from CRAN, loading them with library() or pacman, reading their documentation, and the core dplyr verbs — select, filter, arrange, rename, mutate, summarise — for everyday data manipulation.

A package is a documented bundle of functions, compiled code, and data. Packages exist to make one specific job easy.

Installing a Package

From CRAN

Check the full list of CRAN packages. If a package isn't already installed, install it with `install.packages()`.

```
head(data.frame(installed.packages())[c(1, 3:5)], 5)
```

The installed-packages matrix has a `Package` column, easy to check with `installed.packages()[, "Package"]`:

```
all_packages <- installed.packages()[, "Package"]
"pacman" %in% all_packages
```

Install only if missing:

```
if (!"pacman" %in% all_packages) {
  install.packages("pacman", repos = "http://cran.us.r-project.org")
}
```

Loading a Package

`library()` (from base R) makes a package's functions available:

```
library(pacman)
```

`pacman::p_load()` loads a whole list of packages at once, installing any that are missing:

```
pkg_list <- c("tidyverse", "ggplot2", "lubridate", "flextable")
p_load(pkg_list, character.only = TRUE)
```

Package documentation

```
help(package = "pacman")
```

The full documentation is also available under RStudio's **Files | Plots | Packages | Help** pane.

Suppressing package startup noise

`tidyverse` (and others) print a message on load that you usually don't want in a rendered report:

```
library(tidyverse)
```

The `dplyr` Package

`dplyr` is part of `tidyverse` and provides a consistent grammar for data manipulation, which reads especially well combined with the pipe (`%>%`, covered in Chapter 12).

The most-used `dplyr` verbs are:

- `select()` — keep a subset of columns

```
iris %>% select(1:4) %>% names()
iris %>% select(-Species) %>% names()
```

- `filter()` — keep a subset of rows matching a condition

```
iris %>% filter(Species == "setosa", Petal.Length > 1.4) %>% head(3)
```

- `arrange()` — reorder rows

```
iris %>% arrange(desc(Sepal.Length)) %>% head(3)
```

- `rename()` — rename columns

```
iris %>% rename(PetalLength = Petal.Length) %>% names()
```

- `mutate()` — add or transform columns

```
iris %>% mutate(Ratio = Petal.Length / Petal.Width) %>% head(3)
```

- `summarise()` — collapse a group into one row

```
iris %>% group_by(Species) %>% summarise(mean = mean(Petal.Width), .groups = "drop")
```

▶ View R Markdown

Importing Data in R

Reading built-in datasets, CSVs, and Excel files into R with base R, readr, and rio, then looping over a whole folder of files to load them all at once.

> **Note on file paths:** the original course material referenced files under `./data/`. Those files aren't part of this repo, so every example below either uses a built-in dataset or generates a small temp file on the fly — copy the pattern and point it at your own files.

Built-in Data

R ships with sample datasets ready to use, no import needed.

```
data(iris)
str(iris)
summary(iris)
ncol(iris)
```

Checking the class of every column, timed with base R (or use the `tictoc` package's `tic()` / `toc()` for nicer output if you have it installed):

```
dtypes <- c()
start <- Sys.time()
for (i in 1:ncol(iris)) {
  dtypes[i] <- class(iris[[i]])
}
print(Sys.time() - start)
unique(dtypes)
```

Loading a dataset from a package without loading the whole package:

```
data("spam", package = "kernlab")
str(spam[1:5])
```

CSV Files

Three common ways to read a CSV, from least to most modern:

- `read.csv()` from base R

```
tmp <- tempfile(fileext = ".csv")
write.csv(iris, tmp, row.names = FALSE)

dta <- read.csv(tmp)
head(dta, 3)
```

Notice `Species` comes back as `character` — pass `stringsAsFactors = TRUE` to `read.csv()` if you want it as a factor instead.

- `read_csv()` from `readr` (loaded together with `tidyverse`)

```
library(readr)
dta <- read_csv(tmp)
head(dta, 3)
```

Always check the messages/warnings `read_csv()` prints — they usually flag real issues worth fixing.

- `import()` from `rio`, which auto-detects the file type

```
library(rio)
head(import(tmp), 2)
unlink(tmp)
```

Excel files

`rio::import()` handles `.xlsx` the same way it handles CSV:

```
tmp_xlsx <- tempfile(fileext = ".xlsx")
export(head(iris, 5), tmp_xlsx)
head(import(tmp_xlsx), 2)
unlink(tmp_xlsx)
```

Exercise: import every CSV in a folder with a loop

```
dir.create("demo_csv", showWarnings = FALSE)
write.csv(head(iris, 3), "demo_csv/iris_part.csv", row.names = FALSE)
write.csv(head(mtcars, 3), "demo_csv/mtcars_part.csv", row.names = FALSE)

file_names <- dir("demo_csv", pattern = "\\.csv$")
file_names

data_list <- list()
for (f in file_names) {
  data_list[[f]] <- read.csv(file.path("demo_csv", f))
}
str(data_list)
unlink("demo_csv", recursive = TRUE)
```

▶ **View R Markdown**

The Pipe & Data Manipulation

Chaining steps left-to-right with %>% and the native |> pipe, reshaping data wide-to-long and back with reshape2/tidyr, and producing polished tables with flextable.

The Pipe

Two pipe operators are common in R:

- `%>%` from the `magrittr` package (loaded with `tidyverse` / `dplyr`)
- `|>` the base-R native pipe

RStudio shortcut: **Ctrl+Shift+M**.

Why bother? Because `f(g(h(x)))` reads inside-out, while `x %>% h() %>% g() %>% f()` reads left to right, in the order things actually happen.

```
library(dplyr)
library(ggplot2)

iris %>% group_by(Species) %>%
  summarise(mean = mean(Petal.Width), .groups = "drop")

Gender <- factor(c("Female", "Male", "Male", "Female", "Female", "Male"))

Gender %>% table() %>% data.frame() %>%
  rename("Gender" = 1, Count = Freq) %>%
  mutate(Rel_freq = round(Count / sum(Count), 2)) %>%
  ggplot(aes(x = Gender, y = Rel_freq)) + geom_col()
```

Without the pipe, the same computation nests inside-out:

```
summarise(group_by(iris, Species), mean = mean(Petal.Width))
```

Data Manipulation with `tidyverse`

Loading `tidyverse` pulls in several packages at once: `ggplot2`, `tibble`, `tidyr`, `readr`, `purrr`, `dplyr`, `stringr`, `forcats`, and `lubridate`. There's no need to load each individually afterward.

`dplyr`

Grammar-based data manipulation — see Chapter 10 for the core verbs (`select`, `filter`, `arrange`, `rename`, `mutate`, `summarise`).

`tibble`

A tibble is a modernized data frame: cleaner printing, no silent partial matching, and no automatic factor conversion.

```
library(tibble)
tb <- as_tibble(iris)
tb
glimpse(tb)
```

`reshape2`

Reshaping data between wide and long formats:

```
library(reshape2)
library(tidyr)

view1 <- iris %>% group_by(Species) %>%
  summarise_if(is.numeric, list(mean = mean, sd = sd)) %>%
  melt(id.vars = "Species") %>%
  separate(variable, into = c("variable", "metric"), sep = "_")

view1
```

Pivoting back to wide format with `tidyr::pivot_wider()`:

```
view2 <- view1 %>%
  pivot_wider(names_from = metric, values_from = value) %>%
  mutate(std_error = sd / sqrt(nrow(iris)))
view2
```

```
view2 %>% ggplot() +
  geom_bar(aes(x = Species, y = mean), stat = "identity", fill = "forestgreen", alpha = 0.5) +
  geom_errorbar(aes(x = Species, ymin = mean - std_error, ymax = mean + std_error),
                width = 0.4, colour = "orange", alpha = 0.9, linewidth = 0.5) +
  facet_wrap(~variable, ncol = 4)
```

Presenting Tables

`kableExtra`

Great for styling `knitr::kable()` tables in HTML/PDF reports — see the kableExtra cookbook chapter.

`flextable`

Produces polished tables that also render well in Word documents:

```
library(broom)
library(flextable)

set.seed(123)
x <- rnorm(100, mean = 5, sd = 1.5)
z <- 4 + 8 * x + rnorm(100)
LR <- lm(z ~ x)

tidy(LR) %>% select(-statistic) %>% flextable() %>% autofit() %>% theme_box()
```

See the Epi R Handbook tables chapter for more `flextable` recipes.

`officer`

Generates and edits Word/PowerPoint files directly from R — see the officeverse guide for details.

▶ View R Markdown

Chapter 13 · 14 of 17

Visualization: ggplot2 & plotly

Building layered grammar-of-graphics plots with ggplot2, then making them interactive with plotly and publication-ready with ggpubr.

`ggplot2`

`ggplot2` builds plots in layers, following the "grammar of graphics."

```
library(ggplot2)
library(dplyr)

iris %>%
  ggplot(aes(y = Petal.Length, fill = Species)) +
  geom_boxplot() +
  facet_grid(~Species) +
  theme_minimal()
```

For a full systematic introduction to `ggplot2`, see the Data Visualization chapters of R for Data Science or this ggplot2 bookdown guide.

`plotly`

`plotly` turns static plots into interactive, hover-able ones:

```
library(plotly)

plot_ly(data = iris %>% count(Species),
        labels = ~Species, values = ~n, type = "pie")
```

`ggpubr::ggpie()` gives a similar publication-ready pie chart built on `ggplot2`:

```
library(ggpubr)

ggpie(data = iris %>% count(Species) %>%
        mutate(label = paste0(Species, " (", round(100 * n / sum(n), 2), "%)")),
      x = "n", label = "label", color = "white",
      fill = "Species", lab.pos = "in")
```

See the plotly R docs and this intro-to-DS plotting chapter for more.

▶ View R Markdown

Chapter 14 · 15 of 17

Regular Expressions

Detecting, extracting, and replacing text patterns with base R's grepl/grep/sub/gsub and the more consistent stringr equivalents.

A regular expression (regex) is a pattern used to match, extract, or replace text. R's base functions for this all take a pattern as their first serious argument and a character vector to search as the second.

```
text <- c("dave@example.com", "call 555-1234", "no match here", "ID: R2024-001")
```

Detecting Matches

`grepl()` returns `TRUE` / `FALSE` for each element:

```
grepl("[0-9]+", text)
```

`grep()` returns matching positions, or the matching strings themselves with `value = TRUE`:

```
grep("[0-9]+", text)
grep("[0-9]+", text, value = TRUE)
```

Replacing Matches

`sub()` replaces only the first match per string; `gsub()` replaces all:

```
sub("[0-9]+", "#", text)
gsub("[0-9]+", "#", text)
```

Extracting Matches

`regexpr()` finds the *first* match's position; `regmatches()` pulls out the actual matched text:

```
m <- regexpr("[0-9]+", text)
regmatches(text, m)
```

`gregexpr()` + `regmatches()` extracts *every* match per string:

```
mg <- gregexpr("[0-9]+", text)
regmatches(text, mg)
```

A Practical Pattern: Validating an Email

```
grepl("^[A-Za-z]+@[A-Za-z]+\\.[a-z]+$", text)
```

Common Building Blocks

Pattern	Matches
`[0-9]` or `\\d`	any digit
`[a-zA-Z]`	any letter
`.`	any single character
`+`	one or more of the previous token
`*`	zero or more of the previous token
`^` / `$`	start / end of the string
`\\s`	whitespace
`|`	alternation ("or")

`stringr` Equivalents

The `stringr` package (part of `tidyverse`) wraps the same regex engine in more consistent, pipe-friendly functions:

```
library(stringr)
str_detect(text, "[0-9]+")
str_extract(text, "[0-9]+")
str_extract_all(text, "[0-9]+")
str_replace_all(text, "[0-9]+", "#")
```

For deeper coverage, see DataCamp's regex in R tutorial or this regex tutorial.

▶ View R Markdown

Chapter 15 · 16 of 17

Intro to Modeling: PCA, Clustering, KNN & Regression

A first tour of unsupervised and supervised learning: PCA and clustering to find structure with no labels, then KNN and logistic regression to actually predict a category.

A first tour of the modeling techniques R is most commonly used for, building on the eigen-decomposition and linear model work from earlier chapters.

Principal Component Analysis

Chapter 3 walked through PCA by hand: center the data, eigen-decompose its covariance matrix, and project onto the top eigenvectors. In practice, `prcomp()` does all of that for you, plus proper scaling:

```
data(iris)
pca <- prcomp(iris[, -5], center = TRUE, scale. = TRUE)
summary(pca)
```

`Proportion of Variance` shows how much information each component captures — here the first two components already explain most of the spread in the data.

```
plot(pca$x[, 1:2], col = iris$Species, pch = 16,
     xlab = "PC1", ylab = "PC2", main = "iris projected onto PC1/PC2")
legend("topright", legend = levels(iris$Species),
       col = 1:3, pch = 16, bty = "n")
```

Clustering

K-means

Partitions data into `k` groups by minimizing within-cluster distance. Being unsupervised, it never sees `Species` — comparing its output to the real labels afterward is just for evaluation.

```
set.seed(1)
km <- kmeans(iris[, -5], centers = 3, nstart = 10)
table(cluster = km$cluster, species = iris$Species)
```

Hierarchical clustering

Builds a tree of nested clusters instead of a single flat partition:

```
d <- dist(iris[1:20, -5]) # a small subset, so the dendrogram stays readable
hc <- hclust(d, method = "complete")
plot(hc, main = "Hierarchical clustering (first 20 flowers)")

cutree(hc, k = 3) # cut the tree into 3 groups
```

K-Nearest Neighbor (KNN)

A supervised classifier: label a new point by majority vote among its `k` closest neighbors in the training data.

```
library(class)
set.seed(1)
idx <- sample(1:nrow(iris), 0.7 * nrow(iris))
train <- iris[idx, ]
test  <- iris[-idx, ]

pred <- knn(train[, -5], test[, -5], cl = train$Species, k = 5)
mean(pred == test$Species) # accuracy on the held-out test set
table(predicted = pred, actual = test$Species)
```

Simple Linear Regression

Already covered in Chapter 6's simulated-data example (`z ~ x`, fitted with `lm()`). A quick recap on `iris`:

```
lr <- lm(Petal.Length ~ Petal.Width, data = iris)
coef(lr)
```

Each additional cm of petal width is associated with roughly 2.23 cm more petal length.

Logistic Regression

For a categorical (here, binary) outcome, swap `lm()` for `glm()` with `family = binomial`. Predicting *virginica* vs. *versicolor* from sepal measurements:

```
sub <- subset(iris, Species %in% c("versicolor", "virginica"))
sub$is_virginica <- factor(ifelse(sub$Species == "virginica", 1, 0))

logit <- glm(is_virginica ~ Sepal.Length + Sepal.Width, data = sub, family = binomial)
summary(logit)$coefficients
```

Turning predicted probabilities into class predictions at a 0.5 threshold:

```
pred_prob  <- predict(logit, type = "response")
pred_class <- ifelse(pred_prob > 0.5, 1, 0)

mean(pred_class == sub$is_virginica) # accuracy
table(predicted = pred_class, actual = sub$is_virginica)
```

▶ **View R Markdown**

LaTeX in R Markdown

Writing inline and display math alongside your R code — a natural bridge into Part 3, where LaTeX becomes the main event.

R Markdown (and Quarto) render LaTeX math directly, both inline and as display equations — useful for writing up the statistics behind a model alongside the code that fits it.

Inline Math

Wrapping an expression in single dollar signs keeps it inline with text: the variance of a real-valued variable $X = (X_1, \ldots, X_n)$ is written right in the sentence like this.

Source:

```
The variance of a real-valued variable $X = (X_1, \ldots, X_n)$ is given by:
```

Display Math

Double dollar signs (or a `$$...$$` block) center the equation on its own line:

$$\text{Var}(X) = \left[\frac{1}{n}\sum_{i=1}^n\left(X_i-\frac{1}{n}\sum_{i=1}^nX_i\right)^2\right]^\frac{1}{2}$$

Source:

```
$$
\text{Var}(X) =
\left[\frac{1}{n}\sum_{i=1}^n\left(X_i-\frac{1}{n}
\sum_{i=1}^nX_i\right)^2\right]^\frac{1}{2}
$$
```

Mixing LaTeX and R

Math and code can sit side by side in the same document — describe a function mathematically, then define and run it:

$$f(x) = x^3$$

```
cube <- function(x) x^3
cube(3)
```

This is the same LaTeX engine used to render PDF output (via `tinytex`, set up in Chapter 0), so anything that displays correctly here will also render correctly in a knitted PDF.

▶ View R Markdown



Python Programming

17 lessons · Instructor: Dr. Lema Logamou Seknewna

17 chapters that mirror the R course, from installing Python to a first tour of PCA, clustering and regression. Each chapter is a notebook-style file you can open in Jupyter, VS Code or Colab, and every example was run before it was included.

Chapter 00 · 1 of 17

Getting Started

Installing Python, creating a virtual environment, choosing where to write code (Jupyter, VS Code, Colab) and checking that your tools work.

Python Programming — the Python counterpart of the R course

What is Python?

Python is a general-purpose programming language created by Guido van Rossum and first released in 1991. Its clear, readable syntax and its huge ecosystem (data analysis, machine learning, web, automation) have made it one of the most widely used languages in the world.

Install Python

Download the installer for your system from https://www.python.org/downloads/. On Linux, Python is usually pre-installed; if not: `sudo apt install python3 python3-pip python3-venv`.

Check the installation (in a terminal)

```
python3 --version
pip --version
```

Work in a virtual environment

A virtual environment keeps the packages of one project separate from another.

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install numpy pandas matplotlib scipy scikit-learn jupyterlab
```

Where to write code

- **JupyterLab / Jupyter Notebook** — notebooks mix text, code and results.
- **VS Code** with the Python extension.
- **Google Colab** — nothing to install, runs in the browser.

These chapters are written in the "percent" format: every line that starts with `# %%` begins a new cell. Jupyter (with Jupytext), VS Code and Spyder open them as notebooks, and they also run as ordinary scripts with `python3 chapter.py`. A ready-made `.ipynb` version of each chapter is provided next to it.

```
# Run this cell to check your setup.
import sys

print("Python", sys.version.split()[0])

packages = {"numpy": "numpy", "pandas": "pandas", "matplotlib": "matplotlib",
            "scipy": "scipy", "sklearn": "scikit-learn"}
for module_name, pip_name in packages.items():
    try:
        module = __import__(module_name)
        print(f"{module_name:12s} {module.__version__}")
    except ImportError:
        print(f"{module_name:12s} NOT INSTALLED  ->  pip install {pip_name}")
```

```
print("Hello, Python!")
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/00-getting-started.ipynb

---

Chapter 01 · 2 of 17

Operators

Arithmetic, comparison and logical operators, plus membership tests with in and index() — the small building blocks of every program.

Python can be used as a calculator straight away. Every operator below can be typed in a notebook cell, or in the interactive prompt (`python3`).

Arithmetic operators

```
print(1 + 1)        # addition
print(5 - 1)        # subtraction
print(3 * 4)        # multiplication
print(7 / 2)        # true division: always a float -> 3.5
print(1035 // 3)    # integer (floor) division -> 345
print(7 % 2)        # modulus (remainder) -> 1
print(2 ** 10)      # exponent -> 1024
```

Floor division and modulus follow the sign of the divisor, so `a == (a // b) * b + a % b` always holds:

```
print(-7 // 2, -7 % 3)   # -4  2
print((-7 // 2) * 2 + (-7 % 2))   # -7
```

Comparison operators

Each comparison returns `True` or `False`.

```
print(1 < 1)
print(1 <= 1)
print(1 > 1)
print(1 >= 1)
print("R" == "r")     # Python is case-sensitive
print(1 != 1)
print(1 < 2 < 3)      # comparisons can be chained
```

Logical operators

In plain Python the logical operators are the words `and`, `or`, `not`.

```
print(not True)
print(not (True and False))
print(True and True, True and False, False and False)
print(True or True, True or False, False or False)
```

On NumPy arrays and pandas columns, use the symbols `&` (and), `|` (or) and `~` (not) to combine conditions element by element, with parentheses around each condition.

```
import numpy as np

scores = np.array([45, 72, 88, 61, 95])
print((scores > 60) & (scores < 90))   # between 60 and 90
print((scores < 50) | (scores > 90))   # very low or very high
print(~(scores > 60))                  # not above 60
```

Membership and value matching

`in` tells you whether a value is present. `list.index()` gives its position — but remember that Python counts positions **from 0**.

```
columns = ["Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width", "Species"]

print("Species" in columns)          # True
print("Weight" not in columns)       # True
print(columns.index("Species"))      # 4  (the 5th element)
```

For many values at once on an array, use `np.isin`:

```
print(np.isin(["Species", "Weight"], columns))   # [ True False]
```

▶ **View Python code**

Also available as a Jupyter notebook: python/notebooks/01-operators.ipynb

---

Chapter 02 · 3 of 17

Objects, Assignment & Data Types

Assigning names, the rules for valid names, Python's core types (int, float, complex, str, bool, bytes, None, categories) and how to convert safely between them.

Assignment

Python uses a single assignment operator, `=`. A name:

- may contain letters, digits and underscores, but cannot start with a digit;
- is case-sensitive (`age` and `Age` are different);
- cannot be a reserved word such as `for`, `if`, `class`.

```
a = 1
b = 2
a1 = a              # a new name for the same value
x, y = 10, 20       # assign several names at once
x, y = y, x         # swap two values
print(a, b, a1, x, y)

count = 5
count += 1          # shorthand for count = count + 1
print(count)
```

```
import keyword

print(keyword.kwlist[:10], "...")

# An invalid name raises a SyntaxError. We catch it here so the notebook goes on.
try:
    compile("v rsion = 4", "<demo>", "exec")      # a space inside a name
except SyntaxError as err:
    print("SyntaxError:", err.msg)
```

Data types

Every value has a type. `type()` shows it and `isinstance()` tests it.

int and float (R: integer and numeric)

```
n = 55
pi_value = 3.14159
print(type(n), type(pi_value))
print(2 ** 100)          # Python integers can be arbitrarily large
print(0.1 + 0.2)         # floats are approximate: 0.30000000000000004
```

complex

The imaginary unit is written `j`: `9 + 3j`.

```
z = 9 + 3j
print(z, type(z), z.real, z.imag)

z1 = a + 1j * b           # build a complex number from two real ones
print(z1, type(z1))
```

str (R: character)

```
text = "I am learning Python"
print(text, type(text), len(text))
print("LeaRning" == "Learning")      # case matters
print(text.upper(), "|", text.split())
```

bool (R: logical)

```
v = True
w = False
print(type(v), not v, v and w)
print(isinstance(True, int), True + True)   # bool is a kind of int: True == 1
```

bytes (R: raw)

```
raw_text = "I am learning Python.".encode("utf-8")
print(raw_text[:12], type(raw_text))
print(raw_text.decode("utf-8"))
```

None

`None` represents "no value" (similar to R's `NULL`).

```
result = None
print(result, result is None, type(result))
```

Categories (R: factor)

For categorical data, pandas provides a `category` type.

```
import pandas as pd

gender = pd.Series(["Female", "Male", "Male", "Female"], dtype="category")
print(gender.dtype)
print(gender.cat.categories.tolist())
```

Conversions

Convert with the type's own name: `str()`, `int()`, `float()`, `bool()`.

```
print(str(55), type(str(55)))           # number -> text
print(int("123") + 1)                   # text -> integer
print(float("3.9"))                     # text -> float
print(int(3.9))                         # float -> int TRUNCATES: 3
print(bool(0), bool(""), bool("a"))     # empty/zero values are False
```

A string that is not a valid number raises an error instead of silently becoming a missing value (unlike R). Handle it with `try` / `except`:

```
try:
    int("3.9")        # not a valid integer literal
except ValueError as err:
    print("ValueError:", err)

# pandas can instead turn bad values into NaN, like R's NA:
print(pd.to_numeric("abc", errors="coerce"))
print(pd.to_numeric(pd.Series(["10", "x", "30"]), errors="coerce").tolist())
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/02-objects-assignment-data-types.ipynb

---

Chapter 03 · 4 of 17

Sequences, Arrays & Matrices

NumPy arrays, from sequences and random samples to matrix algebra — transpose, inverse, linear systems, eigenvalues — finishing with a hand-built PCA on the iris data.

Python's built-in `list` can hold anything, but for numerical work we use **NumPy** arrays: fast, one type per array, and arithmetic works on the whole array at once (like R vectors and matrices).

```
import numpy as np

rng = np.random.default_rng(123)    # reproducible randomness (R: set.seed)
```

One-dimensional arrays

```
mixed = [1, "R", True, None]                 # a list may mix types
print(mixed)

numbers = np.array([1, 4, 8, False, True])    # an array has ONE type: everything -> int
print(numbers, numbers.dtype)

words = np.array([1, "R", True])              # ... or text, if a string is present
print(words, words.dtype)
```

Sequences

```
x = np.linspace(0, 2 * np.pi, 90)             # 90 equally spaced points
print(len(x), round(np.diff(x)[0], 5))        # the common step
print(x[:3], x[-3:])                          # first / last values (indices start at 0)
print(x.min(), x.max(), np.ptp(x))            # range = max - min

print(np.arange(0, 10, 2))                    # 0, 2, 4, 6, 8 (end is excluded)
```

Repetition

```
print(np.repeat("I learn Python", 2))
print(np.tile([0, 1], 5))              # 0 1 0 1 0 1 0 1 0 1
print(np.repeat([0, 1], 3))            # 0 0 0 1 1 1   (like rep(each = 3))
```

Random sampling and counting

```
y = rng.choice([0, 1], size=200, p=[1/3, 2/3])
values, counts = np.unique(y, return_counts=True)
print(dict(zip(values.tolist(), counts.tolist())))
print(counts / counts.sum())           # proportions
print(np.mean(y == 0))                 # the mean of a boolean array is a proportion
```

A small survey with typing mistakes

```
from collections import Counter

gender = ["Female", "Male", "Male", "Male", "Female", "Female", "Male", "Femaale",
          "Male", "Female", "Malle", "Male", "Female", "Femmale", "Female", "Male"]
print(Counter(gender))                       # the table, typos included

fixes = {"Femaale": "Female", "Femmale": "Female", "Malle": "Male"}
gender = [fixes.get(g, g) for g in gender]   # replace known typos
print(Counter(gender))
```

Array arithmetic (broadcasting)

```
k = 2
v = np.array([1, 0, 3])
print(k + v, k - v, k * v, v / k, k / np.where(v == 0, np.nan, v))
print(np.array([np.pi, 0, 1, 4]) ** 0.5)
print(np.array([1, 2, 3]) + np.array([10, 20, 30]))   # element by element
```

Matrices (two dimensions)

NumPy fills a matrix **row by row**; R fills it column by column. Pass `order="F"` to `reshape` for R's behaviour.

```
A5 = np.arange(1, 26).reshape(5, 5)              # filled by rows
print(A5)
print(np.arange(1, 26).reshape(5, 5, order="F"))  # filled by columns, like R
```

```
M = np.array([[1, 5, 4],
              [0, 2, 2],
              [2, 1, 0]])
print(M.shape)                      # (3, 3)
print(np.zeros((2, 4)))             # a zero matrix

v1, v2, v3 = np.array([1, 0, 2]), np.array([5, 2, 1]), np.array([4, 2, 0])
print(np.column_stack([v1, v2, v3]))      # R: cbind
print(np.vstack([v1, v2, v3]))            # R: rbind
```

Matrix operations

```
A = M
B = np.array([[2, 3, 0],
              [5, 1, 1],
              [2, 1, 1]])

print(A.T)             # transpose
print(A + B)           # element-wise sum
print(A - B)
print(A * B)           # element-wise product (NOT the matrix product)
print(A @ B)           # matrix product
```

```
A_inv = np.linalg.inv(A)
print(np.round(A_inv, 3))
print(np.round(A @ A_inv))        # the identity matrix
```

Solving a system of equations

$$\begin{cases} 2x + 2y = 0 \\ x + 3y = 2 \end{cases}$$

```
A1 = np.array([[2, 2],
               [1, 3]])
b = np.array([0, 2])
print(np.linalg.solve(A1, b))     # x = -1, y = 1
```

Eigenvalues and eigenvectors

The eigenvalues λ of a square matrix A solve $\det(A - \lambda I) = 0$. They are the basis of Principal Component Analysis.

```
S = np.array([[2., -1., 0.],
              [-1., 2., -1.],
              [0., -1., 2.]])          # a symmetric matrix
values, vectors = np.linalg.eigh(S)     # eigh: for symmetric matrices
print(np.round(values, 4))
print(np.round(vectors.T @ S @ vectors, 4))   # a diagonal matrix of the eigenvalues
```

Worked exercise: PCA by hand on the iris data

1. Keep the four numeric columns.
2. Centre every column.
3. Eigen-decompose `X.T @ X`.
4. Project onto the first two eigenvectors.

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

X = load_iris().data                        # 150 flowers x 4 measurements
col_means = X.mean(axis=0)
print(np.round(col_means, 3))

Xc = X - col_means                          # centred data
cov = Xc.T @ Xc
eigvals, eigvecs = np.linalg.eigh(cov)     # eigenvalues in ascending order
order = np.argsort(eigvals)[::-1]          # sort from largest to smallest
eigvals, eigvecs = eigvals[order], eigvecs[:, order]
print(np.round(eigvals, 3))

projection = Xc @ eigvecs[:, :2]
print(np.round(projection[:3], 3))
```

This is exactly what scikit-learn's `PCA` does (the sign of an axis is arbitrary, so we compare absolute values):

```
scores = PCA(n_components=2).fit_transform(X)
print(np.allclose(np.abs(projection), np.abs(scores)))   # True
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/03-arrays-and-matrices.ipynb

---

Chapter 04 · 5 of 17

Lists, Dictionaries & DataFrames

Lists, tuples, dictionaries and sets, multi-dimensional arrays, and pandas DataFrames — including turning a continuous age into readable age groups.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(123)
```

Multi-dimensional arrays

NumPy arrays can have any number of dimensions. A classic use is gridded data such as maps over time: `array[time, latitude, longitude]`.

```
arr = np.arange(1, 25).reshape(2, 3, 4)      # 2 time steps, each a 3 x 4 grid
print(arr.shape)
print(arr[1, 0, 0])      # first row, first column of the 2nd grid
print(arr[0])            # the whole first grid
print(arr.mean(axis=0))  # average over the first dimension
```

Lists, tuples, dictionaries and sets

- **list** — ordered and changeable: `[1, 2, 3]`
- **tuple** — ordered, fixed: `(1, 2)`
- **dict** — key -> value pairs (like an R named list)
- **set** — unique values

```
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")
print(fruits, fruits[0], fruits[-1], fruits[1:3], len(fruits))

point = (3, 4)
x, y = point                      # unpacking
print(x, y)

record = {"matrix": np.zeros((2, 2)), "sequence": list(range(1, 6)), "flag": True}
print(record["sequence"], list(record.keys()))
record["text"] = "I am learning Python"      # add a new entry
print(len(record))

print(set([1, 2, 2, 3, 3, 3]))               # {1, 2, 3}
```

DataFrames

A pandas `DataFrame` is a table: rows are observations, columns are variables and each column can have a different type (R: `data.frame`).

Turning a continuous variable into categories

```
n = 120
df = pd.DataFrame({
    "ID": [f"Particip_{i}" for i in range(1, n + 1)],
    "age": rng.choice(np.arange(0, 121), size=n, replace=False),
})
print(df.head())

bins = np.arange(0, 121, 10)
df["age_group"] = pd.cut(df["age"], bins=bins, include_lowest=True)
print(df.head())
```

Exercise: summarise the age groups

1. Counts per age group

```
counts = df["age_group"].value_counts().sort_index()
print(counts)
```

2. A bar chart of the groups

```
fig, ax = plt.subplots(figsize=(8, 4))
counts.plot.bar(ax=ax, color="purple")
ax.set_title("Distribution of ages")
ax.set_ylabel("participants")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

3. A histogram of the raw ages

```
plt.figure(figsize=(7, 4))
plt.hist(df["age"], bins=12, color="pink", edgecolor="white")
plt.title("Histogram of ages")
plt.xlabel("age")
plt.show()
```

4. IDs of the participants aged more than 70 up to 80

```
ids = df.loc[(df["age"] > 70) & (df["age"] <= 80), "ID"]
print(ids.tolist())
```

`groupby` splits the table into one piece per group (R: `split`):

```
groups = {key: part for key, part in df.groupby("age_group", observed=True)}
print(list(groups)[:3])
print(groups[pd.Interval(70, 80)]["ID"].tolist())
```

Creating a DataFrame directly

```
df2 = pd.DataFrame({"x": rng.normal(size=10), "y": rng.poisson(2, size=10)})
print(df2.head(3))
print(df2.dtypes)
print(df2.describe().round(2))
```

▶ **View Python code**

Also available as a Jupyter notebook: python/notebooks/04-lists-dicts-dataframes.ipynb

---

Data Cleaning: Missing Values & Outliers

Finding and filling NaN values, rescuing numbers trapped in messy text, and flagging outliers with the IQR method before imputing them.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Missing values (`NaN`)

Missing numbers are represented by `np.nan` ("not a number"; R: `NA`).

```
x = np.array([np.nan, 1, 2, np.nan, 3, np.nan, 3.55])

print(np.where(np.isnan(x))[0])        # positions of the missing values
print(np.where(x >= 2)[0])             # NaN compares as False, so it drops out
print(np.where(~np.isnan(x))[0])       # positions that are NOT missing
print(np.mean(x), np.nanmean(x))       # NaN spreads through a mean; nanmean ignores it
```

Replace the missing values with the mean of the others:

```
x[np.isnan(x)] = np.nanmean(x)
print(x)
```

pandas has dedicated tools: `isna`, `fillna`, `dropna`, `interpolate`.

```
s = pd.Series([4.0, np.nan, 7.0, np.nan, 10.0])
print(s.isna().sum(), "missing")
print(s.fillna(s.mean()).tolist())
print(s.dropna().tolist())
print(s.interpolate().tolist())
```

Missing values created by conversion

Converting messy text to numbers fails for anything that is not a clean number:

```
raw = pd.Series(["2", "1", "2", "7", "$8", "3", "2.5", "9", "2,7"])
y = pd.to_numeric(raw, errors="coerce")        # bad values become NaN
print(y.tolist())
print(raw[y.isna()].tolist())                  # the entries that failed
```

Clean the offending characters first, then convert:

```
messy = pd.Series(["2", "1", "2", ",7", "$8", "3", "&2.5", "9", "2,7"])
clean = (messy.str.replace(r"[$&]", "", regex=True)      # drop currency / junk symbols
              .str.replace(",", ".", regex=False))       # decimal comma -> dot
print(pd.to_numeric(clean).tolist())
```

Outlier detection: the IQR method

A value is an outlier if it lies outside $$[Q_1 - 1.5 \times IQR,\ Q_3 + 1.5 \times IQR] where IQR = Q_3 - Q_1$$.

```
x = np.array([19.103, 19.632, 22.494, 20.113, 10.5, 22.744, 20.737, 17.976,
              18.901, 192.87, 21.959, 0.001, 20.641, 20.177, 19.111, 22.859,
              207.97, 16.853, 21.122, 19.244, 18.291, 19.651, 18.358, 18.834, 19])
x_original = x.copy()

plt.boxplot(x_original)
plt.title("Before handling outliers")
plt.show()
```

```
q1, q2, q3 = np.percentile(x, [25, 50, 75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"Q1={q1:.3f}  Q3={q3:.3f}  IQR={iqr:.3f}  limits=({lower:.2f}, {upper:.2f})")

is_outlier = (x < lower) | (x > upper)
print(x[is_outlier])                    # the outliers themselves
```

Imputing the outliers

Treat them as missing and fill them with the mean of the remaining values.

```
x[is_outlier] = np.nan
x[np.isnan(x)] = np.nanmean(x)

fig, axes = plt.subplots(1, 2, figsize=(8, 4))
axes[0].boxplot(x_original)
axes[0].set_title("Before")
axes[1].boxplot(x)
axes[1].set_title("After")
plt.tight_layout()
plt.show()
```

No outliers remain after the replacement (see the right-hand panel).

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/05-data-cleaning.ipynb

---

Chapter 06 · 7 of 17

Charts & Distribution Simulations

Matplotlib bar, pie, histogram and scatter plots, then simulating uniform, binomial, normal, exponential and Poisson data and fitting a simple linear model.

We use **matplotlib** for plots and NumPy's random generator for simulation.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.datasets import load_iris

rng = np.random.default_rng(12092024)
```

Basic charts

Bar chart and pie chart

```
frequencies = {"Female": 35, "Children": 25, "Male": 40}
colours = ["purple", "salmon", "pink"]

fig, axes = plt.subplots(1, 2, figsize=(9, 4))
axes[0].bar(frequencies.keys(), frequencies.values(), color=colours)
axes[0].set_title("Bar chart")
axes[1].pie(frequencies.values(),
            labels=[f"{k} ({v})" for k, v in frequencies.items()], colors=colours)
axes[1].set_title("Pie chart")
plt.tight_layout()
plt.show()
```

Histogram and density

```
x = rng.integers(1, 121, size=1000)

fig, axes = plt.subplots(1, 2, figsize=(9, 4))
axes[0].hist(x, density=True, color="lightgrey", edgecolor="white")
axes[0].set_title("Histogram (densities)")
grid = np.linspace(x.min(), x.max(), 200)
axes[1].plot(grid, stats.gaussian_kde(x)(grid), color="salmon")
axes[1].set_title("Density of x")
plt.tight_layout()
plt.show()
```

Scatter plot

```
t = np.linspace(0, 2 * np.pi, 50)
plt.scatter(t, np.sin(t), color="blue")
plt.title("sin(x)")
plt.show()
```

Exercise: `petal length` against `petal width`, coloured by species, with a fitted regression line.

```
iris = load_iris()
width, length = iris.data[:, 3], iris.data[:, 2]
slope, intercept = np.polyfit(width, length, 1)

for k, name in enumerate(iris.target_names):
    mask = iris.target == k
    plt.scatter(width[mask], length[mask], label=name, s=18)
xs = np.array([width.min(), width.max()])
plt.plot(xs, intercept + slope * xs, "b--", linewidth=2)
plt.xlabel("Petal width")
plt.ylabel("Petal length")
plt.legend()
plt.show()
```

Simulating distributions

Uniform distribution

$$f_X(x) = \begin{cases} \frac{1}{b-a} & x \in [a, b] \\ 0 & \text{otherwise} \end{cases}$$

```
u = rng.uniform(size=1000)
plt.hist(u, bins=20, density=True, color="turquoise", edgecolor="white")
plt.axvline(u.mean(), color="blue", linestyle="--", label="Average")
plt.title("Histogram of a uniform distribution")
plt.legend()
plt.show()
```

A Kolmogorov-Smirnov test: a large p-value is consistent with "looks uniform"; a Q-Q plot compares the sample with a normal distribution.

```
ks = stats.kstest(u, "uniform")
print(f"KS statistic = {ks.statistic:.3f}, p-value = {ks.pvalue:.3f}")

fig, ax = plt.subplots(figsize=(4.5, 4))
stats.probplot(u, dist="norm", plot=ax)
plt.show()
```

Binomial distribution

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

```
b = rng.binomial(n=1, p=0.3, size=10000)
values, counts = np.unique(b, return_counts=True)

fig, axes = plt.subplots(1, 2, figsize=(8, 3.5))
axes[0].pie(counts, labels=values)
axes[1].bar(values.astype(str), counts)
plt.tight_layout()
plt.show()
```

Gaussian (normal) distribution

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}$$

```
g = rng.normal(0, 1, size=1000)
plt.hist(g, bins=30, density=True, color="turquoise", edgecolor="white")
grid = np.linspace(-4, 4, 200)
plt.plot(grid, stats.norm.pdf(grid), "r--")
plt.title("Standard normal distribution")
plt.show()
print(round(g.mean(), 3), round(g.std(ddof=1), 3))
```

A simulated linear relationship

$$y_i = \alpha + \beta x_i + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, 1)$$

```
x = rng.normal(5, 1.5, size=1000)
z = 4 + 8 * x + rng.normal(size=1000)       # the true relationship is known

fit = stats.linregress(x, z)
print(f"intercept = {fit.intercept:.2f}, slope = {fit.slope:.2f}, R^2 = {fit.rvalue**2:.3f}")

# prediction for a new value x = 15
print("prediction at x = 15:", round(fit.intercept + fit.slope * 15, 2))

plt.scatter(x, z, s=6, color="darkorange", label="Actual")
xs = np.array([x.min(), 15])
plt.plot(xs, fit.intercept + fit.slope * xs, "b-", label="Fitted line")
plt.scatter([15], [fit.intercept + fit.slope * 15], color="darkgreen", label="Predicted")
plt.legend()
plt.show()
```

Exponential and Poisson distributions

```
e = rng.exponential(scale=1.0, size=1000)
p = rng.poisson(lam=2.5, size=1000)

fig, axes = plt.subplots(1, 2, figsize=(9, 3.8))
axes[0].hist(e, bins=30, density=True, color="turquoise", edgecolor="white")
axes[0].set_title("Exponential")
vals, cnt = np.unique(p, return_counts=True)
axes[1].bar(vals, cnt / cnt.sum(), color="turquoise")
axes[1].set_title("Poisson")
plt.tight_layout()
plt.show()
```

▶ **View Python code**

Also available as a Jupyter notebook: python/notebooks/06-charts-and-distributions.ipynb

---

Chapter 07 · 8 of 17

Flow Control: Conditionals & Loops

if/elif/else, for and while loops, break and continue, worked through a grading system, a dice-streak game and reading a folder of files in one loop.

Python marks blocks with **indentation** (4 spaces) instead of braces.

```
import numpy as np

rng = np.random.default_rng(123)
```

if / elif / else

```
x = 3
if x < 4:
    print(True)
else:
    print(False)

print(True if x < 4 else False)      # one-line conditional expression

values = np.array([3, 4, 6, 7, 0, 4])
print(np.where(values < 4, True, False))    # vectorised version (R: ifelse)
```

Grading example

Categorise each mark: Fail [0, 60), Pass [60, 70), Good Pass [70, 80), Very Good Pass [80, 85), Distinction [85, ...).

```
marks = np.round(rng.normal(60, 15, size=50), 2)

# Approach 1: loop over the values
categories = []
for m in marks:
    if m < 60:
        categories.append("Fail")
    elif m < 70:
        categories.append("Pass")
    elif m < 80:
        categories.append("Good Pass")
    elif m < 85:
        categories.append("Very Good Pass")
    else:
        categories.append("Distinction")
print(list(zip(marks[:5], categories[:5])))
```

```
# Approach 2: vectorised with np.select
conditions = [marks < 60, marks < 70, marks < 80, marks < 85]
labels = ["Fail", "Pass", "Good Pass", "Very Good Pass"]
categories2 = np.select(conditions, labels, default="Distinction")
print(list(categories) == categories2.tolist())      # True: same result
```

for loops

```
for i in range(1, 11):          # 1 .. 10  (the end value 11 is excluded)
    if i % 3 == 0:
        print(i, "is a multiple of 3")
    elif i % 2 == 0:
        print(i, "is a multiple of 2")
    else:
        print(i, "is neither a multiple of 2 nor 3")
```

Exercise 1: odd or even

```
for i in range(1, 11):
    print(i, "is even" if i % 2 == 0 else "is odd")
```

`enumerate` gives the position and the value; `zip` walks through several sequences together:

```
names = ["Ada", "Bob", "Chloe"]
ages = [36, 41, 29]
for position, (name, age) in enumerate(zip(names, ages), start=1):
    print(position, name, age)
```

Exercise 2: import every CSV file in a folder

We first create a small demo folder, then read all its files in a loop.

```
import tempfile
from pathlib import Path

import pandas as pd
from sklearn.datasets import load_iris

folder = Path(tempfile.mkdtemp()) / "demo_csv"
folder.mkdir()
iris = load_iris(as_frame=True).frame
iris.head(3).to_csv(folder / "iris_part.csv", index=False)
iris.tail(3).to_csv(folder / "iris_end.csv", index=False)

data = {}
for path in sorted(folder.glob("*.csv")):        # every .csv file in the folder
    data[path.name] = pd.read_csv(path)
for name, frame in data.items():
    print(name, frame.shape)
```

while loops

```
i = 0
while i <= 10:
    print(i * 2)
    i += 10
```

Exercise: a dice streak game

Roll a die until three 5s appear **in a row**.

```
streak, rolls = 0, 0
while streak < 3:
    rolls += 1
    roll = int(rng.integers(1, 7))
    streak = streak + 1 if roll == 5 else 0
print(f"YOU WON! after {rolls} rolls")
```

`break`, `continue` and `while True`

`while True` with `break` plays the role of R's `repeat`.

```
i = 0
while True:
    i += 1
    if i == 3:
        continue          # skip the rest of this round
    print(i)
    if i >= 5:
        break             # leave the loop
```

Exercise: an interactive multiplication table

`input()` waits for the user, so this needs a live session. Put the code in a function and call it yourself:

```
def multiplication_table():
    while True:
        number = int(input("Enter a number 0-9: "))
        for k in range(1, 11):
            print(f"{number} x {k} = {number * k}")
        if input("Continue? (y/n): ").lower() != "y":
            break

# multiplication_table()      # un-comment to try it in a notebook
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/07-flow-control.ipynb

---

Chapter 08 · 9 of 17

Comprehensions & Vectorisation

List comprehensions, NumPy axes and pandas apply, agg and groupby — the Python way to avoid writing explicit loops, like R’s apply family.

R has the *apply family* to avoid explicit loops. In Python the same job is done by **comprehensions**, `map` / `filter`, NumPy's `axis=` argument and pandas' `apply`, `agg` and `groupby`.

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))
print(df.head(3))
```

Comprehensions (R: sapply / lapply)

```
squares = [n ** 2 for n in range(1, 11)]
print(squares)

even_squares = [n ** 2 for n in range(1, 11) if n % 2 == 0]    # with a condition
print(even_squares)

lengths = {name: len(name) for name in ["Ada", "Bob", "Chloe"]}   # dict comprehension
print(lengths)

print(list(map(lambda v: v + 1, [1, 2, 3])))
print(list(filter(lambda v: v > 1, [1, 2, 3])))
```

Along the axes of an array (R: apply on a matrix)

`axis=0` works down the columns, `axis=1` across the rows.

```
A = np.array([[1, 2],
              [3, 4]])
print(A.sum(axis=0), A.sum(axis=1))
print(np.apply_along_axis(lambda row: row.max() - row.min(), 1, A))
```

DataFrame columns (R: apply / sapply)

```
numeric = df.select_dtypes("number")
print(numeric.apply(np.mean))
print(numeric.agg(["min", "max", "std"]).round(2))
print(numeric.apply(lambda col: col.max() - col.min()))      # the range of every column
```

```
def coef_dispersion(s):
    return s.std() / s.mean()

summary = numeric.apply(lambda s: pd.Series({
    "min": s.min(), "max": s.max(), "std": s.std(),
    "Q1": s.quantile(0.25), "median": s.median(), "Q3": s.quantile(0.75),
    "dispersion": coef_dispersion(s)}))
print(summary.round(3))
```

Statistics by group (R: tapply)

```
print(df.groupby("species")["sepal_length"].mean())
print(df.groupby("species")[["sepal_length", "petal_length"]].agg(["mean", "std"]).round(2))
```

Quantiles of every column (R: vapply)

```
print(numeric.quantile([0, 0.25, 0.5, 0.75, 1]))
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/08-comprehensions-and-vectorization.ipynb

---

Chapter 09 · 10 of 17

Writing Functions in Python

Defining functions with defaults, several return values, *args and **kwargs, then two exercises: replacing missing values and solving a quadratic equation.

Function syntax

```
def function_name(arg1, arg2=default):
    """Docstring: what the function does."""
    ...
    return result
```

```
import numpy as np

rng = np.random.default_rng(1)


def toss_die():
    """Roll one six-sided die."""
    return int(rng.integers(1, 7))


print([toss_die() for _ in range(10)])


def add_one(x):
    return x + 1


i = 1
i = add_one(i)
print(i)
```

Default values, keyword arguments and several results

```
def describe(values, decimals=2):
    """Return (mean, standard deviation) of a sequence."""
    values = np.asarray(values, dtype=float)
    return round(values.mean(), decimals), round(values.std(ddof=1), decimals)


mean, sd = describe([4, 8, 15, 16, 23, 42])         # a function can return several values
print(mean, sd)
print(describe([4, 8, 15, 16, 23, 42], decimals=0))  # a keyword argument

square = lambda v: v ** 2        # a short anonymous function
print(square(7))
```

```
def total(*numbers, **options):
    """*numbers collects extra positional arguments, **options keyword ones."""
    result = sum(numbers)
    return result * options.get("factor", 1)


print(total(1, 2, 3), total(1, 2, 3, factor=10))
```

Exercises

1. Replace missing values with a summary statistic

```
def replace_missing(x, fun=np.nanmean):
    """Replace NaN values in x by fun(x), ignoring NaN. Returns a new array."""
    x = np.asarray(x, dtype=float).copy()
    x[np.isnan(x)] = fun(x)
    return x


test = [1, 2, np.nan, 4, np.nan, 6]
print(replace_missing(test))
print(replace_missing(test, fun=np.nanmedian))
```

2. Solve a quadratic equation and comment on its discriminant

For $ax^2 + bx + c = 0$ the discriminant is $\Delta = b^2 - 4ac$:

- $\Delta > 0$ — two distinct real roots
- $\Delta = 0$ — one repeated real root
- $\Delta < 0$ — two complex conjugate roots

```
import math


def solve_quadratic(a, b, c):
    if a == 0:
        raise ValueError("a must be non-zero for a quadratic equation")
    delta = b ** 2 - 4 * a * c
    if delta > 0:
        print("Two distinct real roots")
        return ((-b + math.sqrt(delta)) / (2 * a), (-b - math.sqrt(delta)) / (2 * a))
    if delta == 0:
        print("One repeated real root")
        return (-b / (2 * a),)
    print("Two complex conjugate roots")
    real, imag = -b / (2 * a), math.sqrt(-delta) / (2 * a)
    return (complex(real, imag), complex(real, -imag))


print(solve_quadratic(1, -3, 2))   # delta > 0
print(solve_quadratic(1, 2, 1))    # delta = 0
print(solve_quadratic(1, 0, 1))    # delta < 0
```

Scope

A variable created inside a function is local to it.

```
counter = 0


def increase():
    counter = 100          # a NEW local variable; the outer one is untouched
    return counter


increase()
print(counter)             # still 0
```

▶ **View Python code**

Also available as a Jupyter notebook: python/notebooks/09-writing-functions.ipynb

---

Chapter 10 · 11 of 17

Modules & Packages

Importing modules, a tour of the standard library, installing packages with pip, and the pandas verbs that mirror R's dplyr.

A **module** is a file of Python code; a **package** is a collection of modules. Python ships with a large *standard library*, and the community provides hundreds of thousands more packages on PyPI.

Importing

```
import math                       # the whole module
from math import sqrt, pi         # only some names
import numpy as np                # with a short alias (a universal convention)

print(math.sqrt(16), sqrt(25), round(pi, 4), np.sqrt(36))
```

A tour of the standard library

```
import random
import statistics
from datetime import date
from pathlib import Path

random.seed(1)
print(random.sample(range(1, 50), 6))                # six lottery numbers
print(statistics.mean([2, 4, 4, 4, 5, 5, 7, 9]), statistics.stdev([2, 4, 4, 4, 5, 5, 7, 9]))
print(date(2026, 11, 10).strftime("%A %d %B %Y"))
print(Path.cwd().name)
```

Finding out what a module offers

```
print([name for name in dir(math) if not name.startswith("_")][:8], "...")
print(math.sqrt.__doc__)          # or: help(math.sqrt)
```

Installing packages

Packages are installed with **pip**, in a terminal (not in Python code):

```
pip install pandas matplotlib scikit-learn
pip list                      # what is installed
pip install --upgrade pandas
```

From a notebook cell you can use `%pip install pandas`.

Check from Python whether a package is available, and which version:

```
import importlib.metadata as metadata
import importlib.util

for package in ["numpy", "pandas", "not_a_real_package"]:
    if importlib.util.find_spec(package):
        print(f"{package:20s} installed, version {metadata.version(package)}")
    else:
        print(f"{package:20s} NOT installed -> pip install {package}")
```

pandas verbs (R: the dplyr verbs)

The data-manipulation vocabulary of R's `dplyr` has a direct counterpart in pandas:

dplyr	pandas
`select()`	`df[["a", "b"]]`, `df.drop(columns=...)`
`filter()`	`df[condition]`, `df.query("...")`
`arrange()`	`df.sort_values(...)`
`rename()`	`df.rename(columns=...)`
`mutate()`	`df.assign(new=...)`
`group_by() + summarise()`	`df.groupby(...).agg(...)`

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))

print(df[["sepal_length", "petal_length"]].head(3))                    # select
print(df.drop(columns="species").columns.tolist())
```

```
print(df[(df["species"] == "setosa") & (df["petal_length"] > 1.4)].head(3))   # filter
print(df.query("species == 'setosa' and petal_length > 1.4").shape)

print(df.sort_values("sepal_length", ascending=False).head(3))                 # arrange
print(df.rename(columns={"petal_length": "PetalLength"}).columns.tolist())     # rename

print(df.assign(ratio=df["petal_length"] / df["petal_width"]).head(3))         # mutate
print(df.groupby("species")["petal_width"].mean().round(3))                    # summarise
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/10-modules-and-packages.ipynb

---

Importing Data

Reading CSV, Excel and JSON files with pandas, built-in datasets, and looping over a folder to combine many files into one table.

> **Note on file paths.** The examples below create small temporary files and read them back, so every cell runs on any computer. To use your own data, replace the temporary path by the path of your file.

```
import tempfile
import time
from pathlib import Path

import pandas as pd
from sklearn.datasets import load_iris

workdir = Path(tempfile.mkdtemp())
```

Built-in data

scikit-learn ships small example datasets that are ready to use.

```
iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))

df.info()                                   # column names, types, missing values
print(df.describe().round(2))
print(df.shape)
```

Checking the type of every column, timed with the standard `time` module:

```
start = time.perf_counter()
dtypes = [str(df[col].dtype) for col in df.columns]
print(f"{time.perf_counter() - start:.6f} seconds")
print(sorted(set(dtypes)))
```

CSV files (R: read.csv / read_csv)

```
csv_path = workdir / "iris.csv"
df.to_csv(csv_path, index=False)            # write
data = pd.read_csv(csv_path)                # read
print(data.head(3))
print(data["species"].dtype)                # text ("object" / "str")

data["species"] = data["species"].astype("category")     # R: stringsAsFactors
print(data["species"].dtype)
```

Useful options of `read_csv`: `sep=";"`, `decimal=","`, `na_values=["NA", "-"]`, `usecols=[...]`, `nrows=100`, `encoding="latin-1"`.

Excel files (needs the `openpyxl` package: `pip install openpyxl`)

```
xlsx_path = workdir / "iris.xlsx"
try:
    df.head(5).to_excel(xlsx_path, index=False)
    print(pd.read_excel(xlsx_path).head(2))
except ImportError:
    print("Install openpyxl first:  pip install openpyxl")
```

JSON files

```
json_path = workdir / "iris.json"
df.head(3).to_json(json_path, orient="records")
print(json_path.read_text()[:120], "...")
print(pd.read_json(json_path).shape)
```

Exercise: import every CSV file in a folder with a loop

```
folder = workdir / "demo_csv"
folder.mkdir()
df.head(3).to_csv(folder / "iris_part.csv", index=False)
df.tail(3).to_csv(folder / "iris_end.csv", index=False)

frames = {}
for path in sorted(folder.glob("*.csv")):
    frames[path.name] = pd.read_csv(path)
    print(path.name, frames[path.name].shape)

combined = pd.concat(frames, names=["file", "row"])     # one table, remembering the file
print(combined.shape)
print(combined.reset_index(level="file").head(4))
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/11-importing-data.ipynb

---

Chapter 12 · 13 of 17

Method Chaining & Data Manipulation

Reading analysis steps left to right with method chaining and .pipe(), reshaping wide to long and back, merging tables and presenting results.

R's pipe (`%>%` or `|>`) lets you read a sequence of steps from left to right. pandas does the same with **method chaining**: every method returns a new DataFrame, so you can add `.` after the previous step. Wrapping the chain in parentheses lets you put each step on its own line.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))
```

A first chain

```
result = (
    df.groupby("species")["petal_width"]
      .mean()
      .round(3)
      .rename("mean_petal_width")
      .reset_index()
)
print(result)
```

Without chaining, the same work needs temporary names (or nested calls that read inside-out):

```
step1 = df.groupby("species")["petal_width"]
step2 = step1.mean()
print(step2.round(3))
```

Frequency table of a categorical variable

`.pipe()` lets you put your own function inside a chain.

```
gender = pd.Series(["Female", "Male", "Male", "Female", "Female", "Male"])

table = (
    gender.value_counts()
          .rename_axis("Gender")
          .reset_index(name="Count")
          .assign(Rel_freq=lambda t: (t["Count"] / t["Count"].sum()).round(2))
)
print(table)

def percent(frame, column):
    return frame.assign(**{column + "_pct": (frame[column] * 100).round(0)})

print(table.pipe(percent, "Rel_freq"))

table.plot.bar(x="Gender", y="Rel_freq", legend=False)
plt.ylabel("Relative frequency")
plt.show()
```

Reshaping: wide to long and back (R: reshape2 / tidyr)

```
stats = df.groupby("species").agg(["mean", "std"])      # columns are (variable, statistic)
print(stats.round(2))

# wide -> long: one row per species / variable / statistic
flat = stats.copy()
flat.columns = [f"{var}|{stat}" for var, stat in flat.columns]       # e.g. "petal_length|mean"
long = flat.reset_index().melt(id_vars="species", value_name="value")
long[["variable", "metric"]] = long["variable"].str.split("|", expand=True)
long = long[["species", "variable", "metric", "value"]]
print(long.head(6).round(2))

# long -> wide again
wide = long.pivot(index=["species", "variable"], columns="metric", values="value").reset_index()
wide["std_error"] = wide["std"] / np.sqrt(len(df))      # standard error of the mean
print(wide.round(3).head(4))
```

```
fig, axes = plt.subplots(1, 4, figsize=(12, 3.2), sharey=False)
for ax, (variable, part) in zip(axes, wide.groupby("variable")):
    ax.bar(part["species"], part["mean"], yerr=part["std_error"], color="forestgreen", alpha=0.6,
           ecolor="orange", capsize=4)
    ax.set_title(variable)
    ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```

Combining tables: merge (R: left_join)

```
info = pd.DataFrame({"species": ["setosa", "versicolor", "virginica"],
                     "code": ["SET", "VER", "VIR"]})        # a small look-up table
merged = df.merge(info, on="species", how="left")
print(merged.drop_duplicates("species")[["species", "code"]])
```

Presenting tables

- In a notebook the last expression of a cell is displayed as a formatted table; `df.style` adds colours and formats (needs `jinja2`).
- `df.to_markdown()` (needs `tabulate`) gives a Markdown table.
- `df.to_excel("file.xlsx")` and the `python-docx` / `python-pptx` packages create Word and PowerPoint files from Python.

```
print(wide[["species", "variable", "mean"]].head(3).round(2).to_markdown(index=False))
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/12-method-chaining.ipynb

---

Chapter 13 · 14 of 17

Visualization: seaborn & plotly

Statistical graphics with seaborn and interactive charts with plotly — Python’s

counterparts of ggplot2 and plotly in R.

- **seaborn** builds statistical graphics on top of matplotlib — the Python counterpart of `ggplot2` for everyday plots.
- **plotly** makes interactive charts (hover, zoom).

Install them with `pip install seaborn plotly`.

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))
```

seaborn

Describe *which variables go where*; seaborn takes care of the rest.

```
sns.set_theme(style="whitegrid")

sns.boxplot(data=df, x="species", y="petal_length", hue="species", legend=False)
plt.title("Petal length by species")
plt.show()
```

A grid of small plots, one per species (R: `facet_grid`):

```
grid = sns.FacetGrid(df, col="species", height=3)
grid.map_dataframe(sns.histplot, x="petal_length", bins=10)
plt.show()
```

Relationships between several variables at once:

```
sns.pairplot(df, hue="species", height=1.8)
plt.show()
```

plotly

`plotly.express` creates interactive figures in one line. In a notebook `fig.show()` displays them in place; in a script it opens your browser.

```
import plotly.express as px

counts = df["species"].value_counts().rename_axis("species").reset_index(name="n")
fig = px.pie(counts, names="species", values="n", title="Flowers per species")
fig.show()
```

```
fig = px.scatter(df, x="petal_width", y="petal_length", color="species",
                 hover_data=["sepal_length"], title="Hover over the points")
fig.show()
```

Save an interactive figure as a stand-alone web page you can share:

```
import tempfile
from pathlib import Path

html_path = Path(tempfile.mkdtemp()) / "iris_scatter.html"
fig.write_html(html_path, include_plotlyjs="cdn")      # the page loads plotly.js from the internet
print("saved:", html_path.name, round(html_path.stat().st_size / 1e3), "kB")
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/13-visualization.ipynb

---

Chapter 14 · 15 of 17

Regular Expressions

Detecting, extracting and replacing text patterns with the re module and pandas string methods.

A regular expression (regex) is a pattern used to detect, extract or replace text. Python's `re` module provides them, and pandas exposes them through `Series.str`.

```
import re

text = ["dave@example.com", "call 555-1234", "no match here", "ID: R2024-001"]
```

Detecting matches

`re.search` finds a pattern *anywhere* in a string and returns a match object (or `None`).

```
print([bool(re.search(r"[0-9]+", t)) for t in text])
print([t for t in text if re.search(r"[0-9]+", t)])        # keep the matching strings
```

Replacing matches

`re.sub` replaces **every** match; add `count=1` to replace only the first.

```
print([re.sub(r"[0-9]+", "#", t) for t in text])
print([re.sub(r"[0-9]+", "#", t, count=1) for t in text])
```

Extracting matches

`re.findall` returns every match; `match.group()` returns the text of one.

```
print([re.findall(r"[0-9]+", t) for t in text])
print([m.group() if (m := re.search(r"[0-9]+", t)) else None for t in text])
```

Capturing parts of a match with groups

```
m = re.search(r"(?P<user>[\w.]+)@(?P<domain>[\w.]+)", "write to dave@example.com today")
print(m.group("user"), m.group("domain"), m.span())
```

A practical pattern: validating an email address

`re.fullmatch` requires the *whole* string to match.

```
email_pattern = re.compile(r"[A-Za-z0-9._-]+@[A-Za-z0-9-]+\.[A-Za-z]{2,}")

for candidate in ["dave@example.com", "bad.email@@x", "no-at-sign.com", "user@nodot", "a.b@c.org"]:
    print(f"{candidate:20s} {bool(email_pattern.fullmatch(candidate))}")
```

Common building blocks

Pattern	Matches
`[0-9]` or `\d`	any digit
`[a-zA-Z]`	any letter
`.`	any single character
`+`	one or more of the previous token
`*`	zero or more of the previous token
`^` / `$`	start / end of the string
`\s`	whitespace
`a|b`	`a` or `b`

Write patterns as **raw strings** (`r"..."`) so that backslashes are kept.

Regex in pandas (R: stringr)

```
import pandas as pd

s = pd.Series(text)
print(s.str.contains(r"[0-9]+").tolist())
print(s.str.extract(r"([0-9]+)")[0].tolist())            # first number, NaN if none
print(s.str.replace(r"[0-9]+", "#", regex=True).tolist())
print(s.str.findall(r"[0-9]+").tolist())
```

Exercise: standardise messy product codes

Extract only the numeric ID from codes written in different styles.

```
codes = ["PRD-2023-001", "prd2023002", "PRD_2023_003", "Prd 2023 004", "PRD-2023-005"]
ids = [re.search(r"(\d{3})\D*$", c).group(1) for c in codes]       # the last three digits
print(ids)
years = [re.search(r"(20\d\d)", c).group(1) for c in codes]
print([f"PRD-{year}-{i}" for year, i in zip(years, ids)])
```

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/14-regular-expressions.ipynb

---

Intro to Modeling: PCA, Clustering, KNN & Regression

A first tour of scikit-learn: PCA and clustering for structure without labels, then KNN and logistic regression to predict a category.

---

scikit-learn is Python's standard machine-learning library. Every model follows the same pattern: create it, `fit` it on data, then `predict` or `transform`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris()
X, y = iris.data, iris.target
species = iris.target_names
```

Principal Component Analysis

Chapter 03 built PCA by hand; `PCA` does the same, plus scaling.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)
pca = PCA().fit(X_scaled)
print(np.round(pca.explained_variance_ratio_, 3))      # share of the variance per component

scores = PCA(n_components=2).fit_transform(X_scaled)
for k, name in enumerate(species):
    plt.scatter(scores[y == k, 0], scores[y == k, 1], label=name, s=18)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend()
plt.title("iris projected onto PC1 / PC2")
plt.show()
```

The first two components already explain most of the spread of the data.

Clustering

K-means

K-means never sees the species; comparing its groups with the real labels is only for evaluation.

```
from sklearn.cluster import KMeans

km = KMeans(n_clusters=3, n_init=10, random_state=1).fit(X)
print(pd.crosstab(km.labels_, species[y], rownames=["cluster"], colnames=["species"]))
```

Hierarchical clustering

```
from scipy.cluster.hierarchy import dendrogram, fcluster, linkage

subset = X[:20]                                  # a small subset keeps the tree readable
tree = linkage(subset, method="complete")
dendrogram(tree)
plt.title("Hierarchical clustering (first 20 flowers)")
plt.show()
print(fcluster(tree, t=3, criterion="maxclust"))    # cut the tree into 3 groups
```

K-Nearest Neighbours (KNN)

A supervised classifier: a flower gets the majority label of its `k` closest neighbours in the training data.

```
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
knn = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
pred = knn.predict(X_test)
print("accuracy:", round(accuracy_score(y_test, pred), 3))
print(confusion_matrix(y_test, pred))
```

Simple linear regression

Petal length explained by petal width (see also chapter 06):

```
from sklearn.linear_model import LinearRegression

petal_width, petal_length = X[:, [3]], X[:, 2]
lin = LinearRegression().fit(petal_width, petal_length)
print("intercept:", round(lin.intercept_, 3), " slope:", round(lin.coef_[0], 3))
print("R^2:", round(lin.score(petal_width, petal_length), 3))
```

Logistic regression

For a yes/no outcome we use logistic regression. Here: *virginica* (1) versus *versicolor* (0), from the two sepal measurements. scikit-learn applies a regularisation penalty by default;

a very large `C` switches it off in practice and gives the classical logistic regression.

```
from sklearn.linear_model import LogisticRegression

mask = y != 0                                   # keep versicolor and virginica
Xs, ys = X[mask][:, :2], (y[mask] == 2).astype(int)

logit = LogisticRegression(C=1e10, max_iter=5000).fit(Xs, ys)
print("coefficients:", np.round(logit.coef_[0], 3), " intercept:", np.round(logit.intercept_, 3))

prob = logit.predict_proba(Xs)[:, 1]
predicted = (prob > 0.5).astype(int)            # turn probabilities into classes at 0.5
print("accuracy:", round(accuracy_score(ys, predicted), 3))
print(confusion_matrix(ys, predicted))
```

For p-values and confidence intervals, the `statsmodels` package (`pip install statsmodels`) reports a full statistical summary, like R's `summary(glm(...))`.

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/15-intro-to-modeling.ipynb

---

Chapter 16 · 17 of 17

Math & LaTeX in Notebooks

Writing inline and display equations in notebook cells and inside plots — a bridge to Part 3, where LaTeX becomes the main subject.

Jupyter notebooks (and the `# %% [markdown]` cells of these files) render LaTeX mathematics, so you can write the equations behind a model next to the code that fits it.

Inline and display math

Single dollar signs keep a formula inside the sentence: the variance of $X = (X_1, \ldots, X_n)$ is written right in the text.

Double dollar signs put the equation on its own line:

$$\mathrm{Var}(X) = \frac{1}{n}\sum_{i=1}^{n}\left(X_i - \bar{X}\right)^2, \qquad \bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

Equations in code output

`IPython.display.Math` renders a LaTeX string in a notebook cell. (Run as a plain script it only prints an object; in a notebook it shows the formula.)

```
try:
    from IPython.display import Math, display

    display(Math(r"f(x) = x^3"))
except ImportError:                      # IPython is installed with Jupyter
    print(r"f(x) = x^3")
```

Mathematics inside plots

matplotlib understands a subset of LaTeX ("mathtext") in any text, as long as the formula is between dollar signs and the string is a raw string `r"..."`.

```
import matplotlib.pyplot as plt
import numpy as np


def cube(x):
    return x ** 3


x = np.linspace(-2, 2, 100)
plt.plot(x, cube(x), label=r"$f(x)=x^3$")
plt.plot(x, np.sin(x), "--", label=r"$g(x)=\sin(x)$")
plt.title(r"Two functions: $f(x)=x^3$ and $g(x)=\sin x$")
plt.xlabel(r"$x$")
plt.ylabel(r"$y=f(x)$")
plt.legend()
plt.show()

print(cube(3))
```

A fitted model written as an equation

Use the numbers from a fitted model to build the LaTeX text automatically:

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression

iris = load_iris()
fit = LinearRegression().fit(iris.data[:, [3]], iris.data[:, 2])

equation = rf"$\widehat{{\mathrm{{length}}}} = {fit.intercept_:.2f} + {fit.coef_[0]:.2f}\,\mathrm{{width}}$"
print(equation)

plt.figure(figsize=(6, 2))
plt.axis("off")
plt.text(0.5, 0.5, equation, ha="center", va="center", fontsize=18)
plt.show()
```

From notebook to PDF

`jupyter nbconvert --to pdf notebook.ipynb` produces a PDF; it needs a LaTeX distribution such as TeX Live or MiKTeX (see Part 3 of the course). To use full LaTeX for the text inside matplotlib figures, set `plt.rcParams["text.usetex"] = True` (also requires LaTeX).

▶ View Python code

Also available as a Jupyter notebook: python/notebooks/16-math-and-latex-in-notebooks.ipynb

LATEX

LaTeX

22 lessons · Instructor: Dr. Lema Logamou Seknewna

22 sections on typesetting documents by writing, not clicking.

Section 01 · 1 of 22

Introduction

Typesetting documents by writing, not clicking. 22 slides from a blank file to your first properly structured document.

Overleaf's full tutorialTry it online (Overleaf)

Section 02 · 2 of 22 Concepts

What Is LaTeX?

Most word processors are **WYSIWYG** — what you see is what you get. You click, drag, and format a page directly, and the tool shows you the result as you go.

LaTeX works the opposite way. You write a plain text file full of commands that describe *what things are* — a section, a table, an equation — not how they should look pixel-by-pixel. A program called a **TeX engine** then reads that file and compiles it into a finished PDF.

The upshot: you focus on content and structure, and LaTeX handles the visual formatting for you, consistently, every time.

Section 03 · 3 of 22 Concepts

Why Learn LaTeX?

- Handles complex math, technical notation, and large tables far more gracefully than a word processor.
- Numbers figures, equations, citations, and table-of-contents entries automatically — renumber freely without breaking anything.
- Separates content from style: rewrite what a document looks like without touching a word of what it says.
- Extensible through thousands of free, community-written packages.

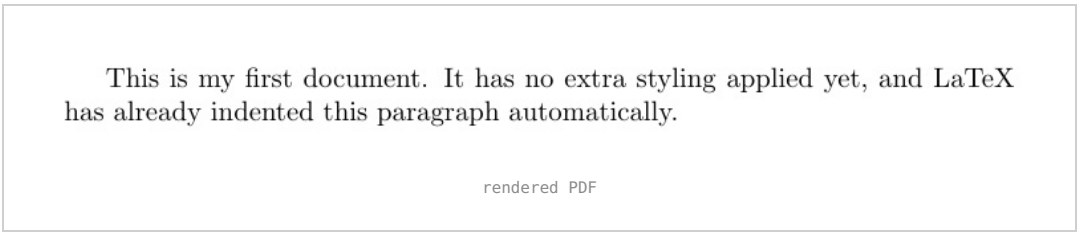
It's not the right tool for everything — free-form magazine-style layouts are still easier in a visual design tool. But for anything technical, academic, or reference-heavy, it earns its learning curve fast.

Section 04 · 4 of 22 Your First Document

Documentclass & Body

main.tex

```
\documentclass{article}
\begin{document}
This is my first document. It has no extra
styling applied yet, and LaTeX has already
indented this paragraph automatically.
\end{document}
```



rendered PDF

`\documentclass` sets the document type. Everything between `\begin{document}` and `\end{document}` is the body — the part that actually gets typeset.

Section 05 · 5 of 22 Structure

The Preamble

Everything *before* `\begin{document}` is called the **preamble** — the document's setup section.

preamble

```
\documentclass[12pt, a4paper]{article}
\usepackage{amsmath}
```

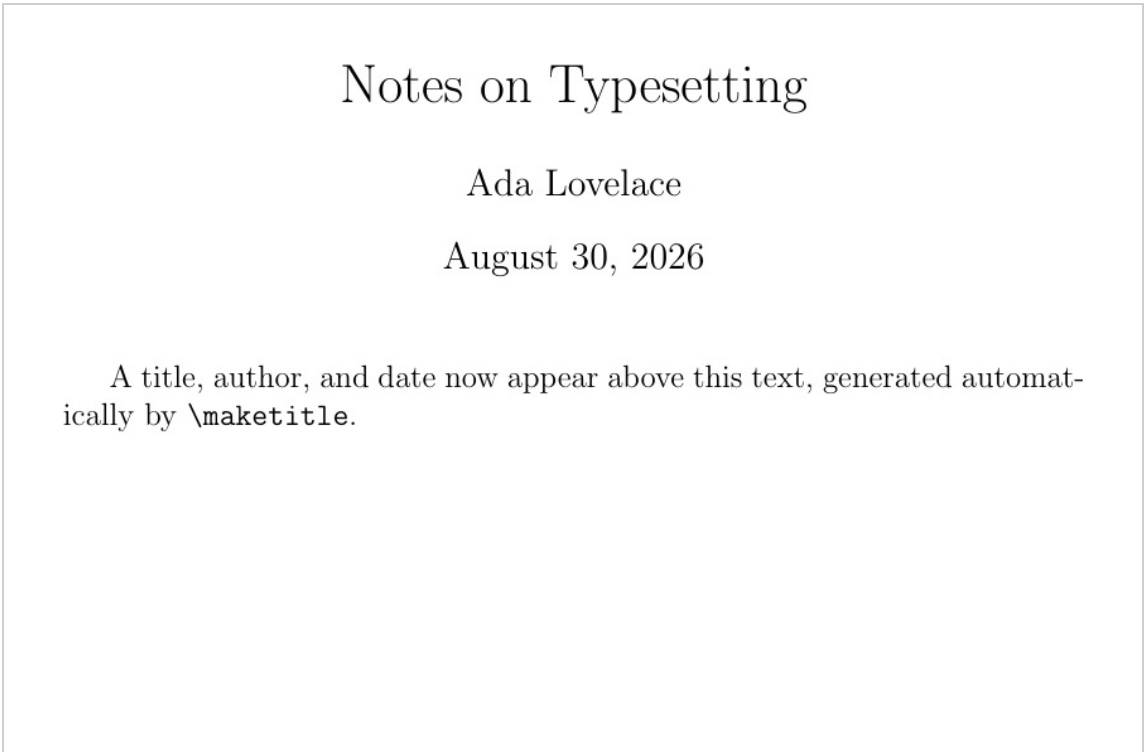
- Options inside `[...]` configure the class — here, 12-point type on A4 paper.
- `\usepackage{...}` loads extra functionality — more on packages in a few slides.

Section 06 · 6 of 22 Structure

Title, Author & Date

main.tex

```
\documentclass[12pt, letterpaper]{article}
\title{Notes on Typesetting}
\author{Ada Lovelace}
\date{\today}
\begin{document}
\maketitle
A title, author, and date now appear above
this text, generated automatically by
\maketitle.
\end{document}
```



1

rendered PDF

`\date{\today}` inserts the actual compile date automatically, every single time you rebuild the document.

Section 07 · 7 of 22 Structure

Comments

main.tex

```
% Everything after a percent sign is ignored.
% Use comments for to-do notes, explanations,
% or to disable a line while debugging.
\documentclass{article}
```

A `%` at the start of a line — or anywhere mid-line — comments out everything after it. Nothing after a `%` gets typeset.

Section 08 · 8 of 22 Formatting

Bold, Italic & Underline

main.tex

```
Some ideas are \textbf{bold}, some
are \textit{subtle}, and some
deserve \underline{extra attention}.
```



rendered PDF

There's also `\emph{...}`, which adapts to context: italic inside upright text, but it flips to upright if you're already inside an italicized block.

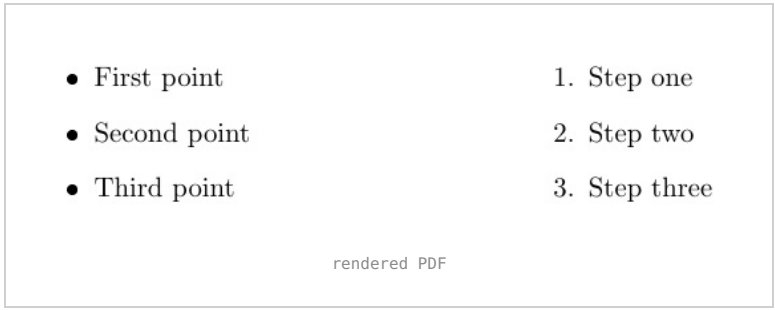
Section 09 · 9 of 22 Formatting

Lists

main.tex

```
\begin{itemize}
  \item First point
  \item Second point
\end{itemize}

\begin{enumerate}
  \item Step one
  \item Step two
\end{enumerate}
```



rendered PDF

`itemize` gives bullets, `enumerate` gives automatic numbering — same `\item` syntax either way.

Section 10 · 10 of 22 Media

Adding Images

Importing a graphic needs the `graphicx` package, loaded in the preamble:

main.tex

```
\usepackage{graphicx}
\graphicspath{{figures/}}

\includegraphics[width=0.75\textwidth]{diagram}
```

- `\graphicspath` tells LaTeX which folder to look in.
- Give the filename without its extension — LaTeX searches supported formats for you.
- `width=0.75\textwidth` scales the image relative to the text column, not an absolute size.

Section 11 · 11 of 22 Media

Captions, Labels & Cross-References

main.tex

```
\begin{figure}[h]
  \centering
  \includegraphics[width=0.6\textwidth]{diagram}
  \caption{A labeled diagram.}
  \label{fig:diagram}
\end{figure}

See Figure~\ref{fig:diagram} on
page~\pageref{fig:diagram}.
```

`\label` + `\ref` is the whole trick: LaTeX tracks the actual figure number for you. Reorder or insert figures freely — every reference updates itself on the next compile.

Section 12 · 12 of 22 Math

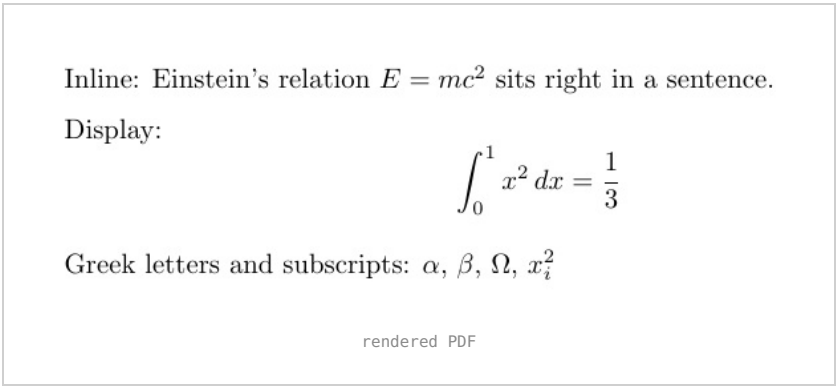
Math Mode: Inline vs. Display

main.tex

```
Inline: Einstein's relation
$E=mc^2$ sits right in a sentence.

Display:
\[ \int_0^1 x^2\,dx = \frac{1}{3} \]

Greek and subscripts:
$\alpha$, $\beta$, $\Omega$, $x_i^2$
```



rendered PDF

Single `$...$` keeps math inline with text. `\[...\]` centers an equation on its own line — prefer it over the old-style `$$...$$`, which modern LaTeX advises against.

Section 13 · 13 of 22 Math

Common Math Building Blocks

Want	Write
Fraction	`\frac{a}{b}`
Subscript / superscript	`a_b` / `a^b`
Square root	`\sqrt{x}`
Integral	`\int_0^1 ... \, dx`
Summation	`\sum_{i=1}^n`
Greek letters	`\alpha \beta \Omega`
Named functions	`\sin \cos \log`

All of these work inside either `$...$` or `\[...\]`. Named functions like `\sin` matter — plain `sin(x)` renders italicized as if it were three separate variables multiplied together.

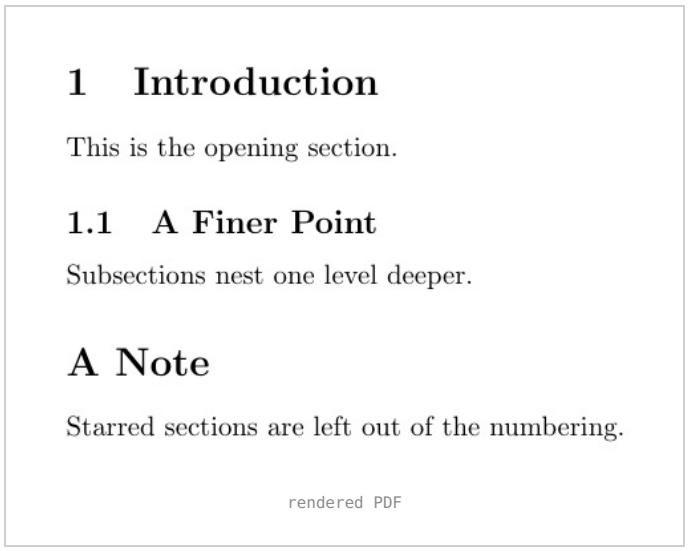
Sections & Chapters

main.tex

```
\section{Introduction}
This is the opening section.

\subsection{A Finer Point}
Subsections nest one level deeper.

\section*{A Note}
Starred sections skip numbering.
```



rendered PDF

`\chapter` and `\part` only exist in the `report` and `book` classes — `article` tops out at `\section`. Numbering is automatic unless you use the starred (`*`) form.

Section 15 · 15 of 22 Document Structure

Paragraphs & Line Breaks

main.tex

```
First paragraph.

Second paragraph starts after a blank line.

Third paragraph continues on\\
a manual new line, same paragraph.
```

- A blank line starts a new paragraph — LaTeX indents it automatically.
- `\\` (or `\newline`) breaks the line without starting a new paragraph.

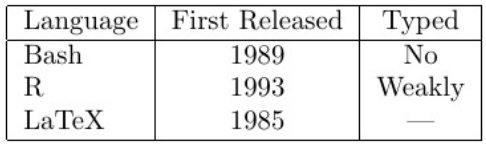
Avoid stacking `\\\\` to fake extra spacing between paragraphs — it fights LaTeX's own spacing rules. Use the `parskip` package instead if you want visible gaps.

Section 16 · 16 of 22 Tables

Building a Table

main.tex

```
\begin{tabular}{|l|c|c|}
\hline
Language & First Released & Typed \\
\hline
Bash & 1989 & No \\
R & 1993 & Weakly \\
LaTeX & 1985 & --- \\
\hline
\end{tabular}
```



Language	First Released	Typed
Bash	1989	No
R	1993	Weakly
LaTeX	1985	—

rendered PDF

`{|l|c|c|}` declares three columns (left/center/center-aligned) with vertical rules between them. `&` separates cells, `\\` ends a row, and `\hline` draws a horizontal rule.

Section 17 · 17 of 22 Document Structure

Table of Contents

main.tex

```
\tableofcontents

\section{Background}
...
\section{Findings}
...
```

One command builds and numbers the entire table of contents from whatever `\section` / `\subsection` commands already exist in your document. Compile twice — the first pass collects the section numbers and page numbers, the second pass writes them into the contents page.

Section 18 · 18 of 22 Packages

Loading Packages & Options

main.tex

```
\usepackage{geometry}
\usepackage[margin=1in]{geometry}
```

Options inside `[...]` configure a package's behavior — here, setting every page margin to 1 inch. Without options, a package just loads with its defaults.

If LaTeX can't find the package's underlying `.sty` file, it stops with a clear "file not found" error rather than silently failing.

Section 19 · 19 of 22 Packages

Finding Packages: CTAN & TeX Live

CTAN (the Comprehensive TeX Archive Network) hosts thousands of free, community-contributed packages covering everything from chemistry notation to chess diagrams.

TeX Live bundles a large, tested snapshot of CTAN into one installable distribution, refreshed annually. Overleaf, and most desktop LaTeX installs, run on TeX Live — so a package can exist on CTAN without yet being available to you if it's very new or has licensing/platform quirks.

Browse CTANAbout TeX Live

Section 20 · 20 of 22 Reference

Cheat Sheet

Command	Does
`\documentclass{article}`	Sets the document type
`\usepackage{name}`	Loads extra functionality
`\section{...}`	Numbered heading
`\textbf{} \textit{}`	Bold / italic
`itemize` / `enumerate`	Bulleted / numbered list
`$...$` / `\[...\]`	Inline / display math
`tabular`	Table environment
`\label` / `\ref`	Cross-reference anything
`\tableofcontents`	Auto-generated contents page

Section 21 · 21 of 22 Keep Going

Where to Go Next

This deck covers the same ground as Overleaf's own free introduction, condensed and restructured as slides — their version has more detail on every topic and is worth reading end to end.

Overleaf: Learn LaTeX in 30 minutesOverleaf documentation home

Beyond the basics: bibliography management (`biblatex` / `bibtex`), the `Beamer` class for building actual slide presentations in LaTeX itself, and `TikZ` for drawing diagrams directly in your document.

Section 22 · 22 of 22

What’s next

Part 4 puts it to work: Mathematical Epidemiology prep in R and Python.



Mathematical Epidemiology Prep

8 lessons · Instructor: Dr. Lema Logamou Seknewna

8 short chapters that prepare students for a course in Mathematical Epidemiology, each written twice, in R and in Python, with the same parameters so the results can be compared line by line. They use only basic examples: growth rates, the SIR and SEIR models, R_0 and herd immunity, vaccination, stochastic outbreaks, and fitting a model to data. Setup: `install.packages("deSolve")` for R, and `pip install numpy scipy matplotlib` for Python.

Chapter 00 · 1 of 8

Toolkit: R and Python Side by Side

The handful of tools every later chapter relies on (functions, vectors, loops and plots), introduced through the basic reproduction number $R_0 = \beta/\gamma$, with an R-to-Python cheat sheet for translating between the two languages.

Models in epidemiology are built from a handful of numbers (rates, populations, time steps) and a handful of operations (functions, loops, plots). This chapter shows the basic R tools you will use, and, in the Python version of this chapter, the same tools in Python.

One idea to practice with: the basic reproduction number

The basic reproduction number R_0 is the average number of people one infectious person infects in a fully susceptible population. In the simplest model

$$R_0 = \frac{\beta}{\gamma}$$

where β is the transmission rate (per day) and γ is the recovery rate (per day), so $1/\gamma$ is the average infectious period.

```
r0 <- function(beta, gamma) beta / gamma

r0(beta = 0.5, gamma = 0.2)   # 2.5: one case infects 2.5 others on average
```

Vectors and loops

```
betas <- c(0.2, 0.4, 0.6, 0.8)
gamma <- 0.25

for (b in betas) {
  cat("beta =", b, " -> R0 =", r0(b, gamma), "\n")
}

r0(betas, gamma)              # functions work on whole vectors at once
```

A data frame of scenarios

```
scenarios <- data.frame(beta = betas, gamma = gamma)
scenarios$R0 <- r0(scenarios$beta, scenarios$gamma)
scenarios$can_spread <- scenarios$R0 > 1   # an outbreak needs R0 > 1
scenarios
```

A first plot

```
plot(betas, r0(betas, gamma), type = "b", pch = 16,
     xlab = "Transmission rate (beta)", ylab = "R0",
     main = "R0 grows with the transmission rate")
abline(h = 1, lty = 2)          # the threshold R0 = 1
```

R and Python cheat sheet

Task	R	Python (numpy)
A vector / array	`c(1, 2, 3)`	`np.array([1, 2, 3])`
Sequence	`seq(0, 1, by = 0.1)`	`np.arange(0, 1.01, 0.1)` or `np.linspace(0, 1, 11)`
Length	`length(x)`	`len(x)`
First element	`x[1]`	`x[0]` (Python counts from 0)
Loop	`for (i in 1:3) { ... }`	`for i in range(3): ...`
Function	`f <- function(x) x^2`	`def f(x): return x**2`
Power	`x^2`	`x**2`
Plot	`plot(x, y)`	`plt.plot(x, y)`

▶ View R Markdown

▼ View Python script

○ ○ ○ mathepi/python/00-toolkit.py

```
"""
```

```
MathEpi 00 - Toolkit: R and Python side by side

Run with:  python 00-toolkit.py
Needs:     numpy, matplotlib   (pip install numpy matplotlib)

The basic reproduction number R0 is the average number of people one
infectious person infects in a fully susceptible population.
In the simplest model:  R0 = beta / gamma
  beta  = transmission rate (per day)
  gamma = recovery rate (per day), so 1/gamma is the infectious period
"""
import numpy as np
import matplotlib.pyplot as plt


# A function (R: r0 <- function(beta, gamma) beta / gamma)
def r0(beta, gamma):
    return beta / gamma


print("R0 =", r0(beta=0.5, gamma=0.2))   # 2.5

# Arrays and loops (R: betas <- c(0.2, 0.4, 0.6, 0.8))
betas = np.array([0.2, 0.4, 0.6, 0.8])
gamma = 0.25

for b in betas:
    print(f"beta = {b} -> R0 = {r0(b, gamma):.1f}")

print(r0(betas, gamma))   # numpy functions work on whole arrays at once

# A small table of scenarios (R: a data frame)
print("\n beta   gamma   R0    can_spread")
for b in betas:
    R0 = r0(b, gamma)
    print(f" {b:.1f}    {gamma:.2f}   {R0:.1f}   {R0 > 1}")   # an outbreak needs R0 > 1

# A first plot
plt.plot(betas, r0(betas, gamma), "o-")
plt.axhline(1, linestyle="--", color="grey")    # the threshold R0 = 1
plt.xlabel("Transmission rate (beta)")
plt.ylabel("R0")
plt.title("R0 grows with the transmission rate")
plt.show()

# ------------------------------------------------------------------
# R vs Python cheat sheet
#   Task            R                        Python (numpy)
#   array           c(1, 2, 3)               np.array([1, 2, 3])
#   sequence        seq(0, 1, by = 0.1)      np.linspace(0, 1, 11)
#   length          length(x)                len(x)
#   first element   x[1]                     x[0]   <- Python counts from 0!
#   loop            for (i in 1:3) { ... }   for i in range(3): ...
#   function        f <- function(x) x^2     def f(x): return x**2
#   power           x^2                      x**2
#   plot            plot(x, y)               plt.plot(x, y)
# ------------------------------------------------------------------
```

Exponential Growth and Doubling Time

Early in an outbreak, cases grow exponentially. Simulate noisy daily case counts, estimate the growth rate with a log-linear fit, and convert it into a doubling time.

At the start of an outbreak, almost everyone is susceptible, so cases grow roughly exponentially:

$$I(t) \approx I_0 \, e^{rt}$$

The growth rate r can be estimated from data, and from it the **doubling time** $T_d = \ln(2)/r$.

Simulate daily case counts

We pretend the true growth rate is $r = 0.15$ per day, starting from 5 cases, and add Poisson noise because real counts are never perfectly smooth.

```
set.seed(1)
r_true <- 0.15
day <- 0:29
cases <- rpois(length(day), lambda = 5 * exp(r_true * day))
data <- data.frame(day, cases)
head(data)
```

Estimate the growth rate

Taking logs turns the exponential curve into a straight line: $\log I(t) = \log I_0 + r\,t$. So a linear regression of `log(cases)` on `day` estimates r as the slope.

```
fit <- lm(log(cases) ~ day, data = data)
r_hat <- unname(coef(fit)["day"])
r_hat                       # close to the true value 0.15
log(2) / r_hat              # estimated doubling time in days
log(2) / r_true             # true doubling time
```

Plot data and fit on a log scale

```
plot(day, cases, log = "y", pch = 16,
     xlab = "Day", ylab = "Daily cases (log scale)")
lines(day, exp(predict(fit)), col = "red", lwd = 2)
```

On a log scale, exponential growth is a straight line. Real outbreaks leave this straight line once susceptibles are depleted or control measures start.

▶ **View R Markdown**

▼ **View Python script**

○ ○ ○ mathepi/python/01-exponential-growth.py

```
"""
MathEpi 01 - Exponential growth and doubling time

At the start of an outbreak almost everyone is susceptible, so cases grow
roughly exponentially:   I(t) ~ I0 * exp(r * t)
The growth rate r can be estimated from data, and the doubling time is
    T_d = ln(2) / r
"""
import numpy as np
import matplotlib.pyplot as plt

# Simulate daily case counts: true r = 0.15 per day, 5 cases on day 0,
# plus Poisson noise (real counts are never perfectly smooth).
rng = np.random.default_rng(1)          # (R: set.seed(1))
r_true = 0.15
day = np.arange(30)                      # days 0..29
cases = rng.poisson(5 * np.exp(r_true * day))
print("first days:", cases[:6])

# Taking logs turns the exponential into a straight line:
#     log I(t) = log I0 + r * t
# so a straight-line fit of log(cases) against day gives r as the slope.
slope, intercept = np.polyfit(day, np.log(cases), 1)
r_hat = slope
print(f"estimated r      = {r_hat:.3f}   (true {r_true})")
print(f"doubling time    = {np.log(2) / r_hat:.2f} days")
print(f"true doubling    = {np.log(2) / r_true:.2f} days")

# Plot data and fit on a log scale (a straight line = exponential growth)
plt.semilogy(day, cases, "o", label="Observed")
plt.semilogy(day, np.exp(intercept + slope * day), "r-", label="Fitted")
plt.xlabel("Day")
plt.ylabel("Daily cases (log scale)")
plt.legend()
plt.show()
```

Chapter 02 · 3 of 8

The SIR Model

The classic three-compartment model: Susceptible, Infectious, Recovered. Solve its differential equations numerically with deSolve in R and solve_ivp in Python, then read off the peak and the total number infected.

The SIR model splits a population of size N into three compartments: **S**usceptible,

Infectious, and **R**ecovered (or removed).

$$\frac{dS}{dt} = -\beta \frac{SI}{N}, \qquad \frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \qquad \frac{dR}{dt} = \gamma I$$

Susceptibles become infectious at rate $\beta SI/N$; infectious people recover at rate γ. The three equations add up to zero, so the total population N is constant.

Solve it numerically with `deSolve`

```
library(deSolve)

sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

N     <- 1000
init  <- c(S = N - 1, I = 1, R = 0)
parms <- c(beta = 0.5, gamma = 0.2, N = N)
times <- seq(0, 120, by = 1)

out <- as.data.frame(ode(y = init, times = times, func = sir, parms = parms))
head(out)
```

Plot the epidemic

```
matplot(out$time, out[, c("S", "I", "R")], type = "l", lty = 1, lwd = 2,
        col = c("blue", "red", "darkgreen"),
        xlab = "Day", ylab = "People", main = "SIR epidemic")
legend("right", c("S", "I", "R"), col = c("blue", "red", "darkgreen"), lty = 1, lwd = 2)
```

Read off the key numbers

```
peak_I   <- max(out$I)
peak_day <- out$time[which.max(out$I)]
c(peak_infectious = round(peak_I), peak_day = peak_day,
  total_ever_infected = round(max(out$R)))
```

The epidemic rises, peaks when $S/N = 1/R_0$, and then burns out, leaving some people who were never infected.

▶ View R Markdown

▼ View Python script

○ ○ ○ mathepi/python/02-sir-model.py

```
"""
```

```
MathEpi 02 - The SIR model

Susceptible -> Infectious -> Recovered, in a population of size N:

    dS/dt = -beta * S * I / N
    dI/dt =  beta * S * I / N - gamma * I
    dR/dt =  gamma * I

Needs: numpy, scipy, matplotlib
"""
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt


def sir(t, y, beta, gamma, N):
    S, I, R = y
    dS = -beta * S * I / N
    dI = beta * S * I / N - gamma * I
    dR = gamma * I
    return [dS, dI, dR]


N = 1000
beta, gamma = 0.5, 0.2
y0 = [N - 1, 1, 0]                      # S, I, R at time 0
days = np.arange(0, 121)                # 0, 1, ..., 120

# rtol sets how accurately the solver works; a tight value gives the same
# numbers as R's deSolve
sol = solve_ivp(sir, (0, 120), y0, args=(beta, gamma, N), rtol=1e-8, t_eval=days)
S, I, R = sol.y

plt.plot(sol.t, S, "b", label="S")
plt.plot(sol.t, I, "r", label="I")
plt.plot(sol.t, R, "g", label="R")
plt.xlabel("Day")
plt.ylabel("People")
plt.title("SIR epidemic")
plt.legend()
plt.show()

print(f"peak infectious     = {I.max():.0f}")
print(f"peak day            = {sol.t[I.argmax()]:.0f}")
print(f"total ever infected = {R[-1]:.0f}")
```

Chapter 03 · 4 of 8

R_0, Herd Immunity and Final Size

Three numbers that summarise an outbreak: R_0, the herd immunity threshold $1 - 1/R_0$, and the final epidemic size, found by solving $z = 1 - e^{-R_0 z}$ numerically and checked against a simulation.

Three quantities summarise most of what the SIR model says about an outbreak.

- **Basic reproduction number:** $R_0 = \beta/\gamma$. An outbreak can take off only if $R_0 > 1$.
- **Herd immunity threshold:** the fraction of the population that must be immune so that $R_0 \times (\text{fraction susceptible}) \le 1$, i.e. $1 - 1/R_0$.
- **Final size:** the fraction z of the population ever infected, which solves $z = 1 - e^{-R_0 z}$ (when almost everyone starts susceptible).

```
beta <- 0.5
gamma <- 0.2
R0 <- beta / gamma
herd_immunity <- 1 - 1 / R0
c(R0 = R0, herd_immunity_threshold = herd_immunity)
```

Solving for the final size

The final-size equation has no closed-form solution, so we find its root numerically with `uniroot()`.

```
final_size <- function(R0) {
  if (R0 <= 1) return(0)                       # no major outbreak
  uniroot(function(z) z - (1 - exp(-R0 * z)),
          lower = 1e-6, upper = 1)$root
}

final_size(R0)
```

Check against the simulated epidemic

```
library(deSolve)
sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}
N <- 1000
out <- as.data.frame(ode(c(S = N - 1, I = 1, R = 0), seq(0, 300, by = 1),
                         sir, c(beta = beta, gamma = gamma, N = N)))
tail(out$R, 1) / N            # fraction ever infected in the simulation
```

The simulation and the equation agree.

How the final size depends on R0

```
R0_values <- c(1.2, 1.5, 2, 2.5, 3, 4)
table_out <- data.frame(
  R0 = R0_values,
  herd_immunity = round(1 - 1 / R0_values, 3),
  final_size = round(sapply(R0_values, final_size), 3)
)
table_out
```

```
curve_R0 <- seq(1.05, 5, by = 0.05)
plot(curve_R0, sapply(curve_R0, final_size), type = "l", lwd = 2,
     xlab = "R0", ylab = "Fraction ever infected",
     main = "Final epidemic size")
lines(curve_R0, 1 - 1 / curve_R0, lty = 2)
legend("bottomright", c("Final size", "Herd immunity threshold"), lty = c(1, 2))
```

Notice that the final size is always larger than the herd immunity threshold: the epidemic overshoots because infections continue while I is still large.

▶ View R Markdown

▼ View Python script

○ ○ ○ mathepi/python/03-r0-herd-immunity.py

```
"""
MathEpi 03 - R0, herd immunity and final size

  R0                = beta / gamma         (an outbreak needs R0 > 1)
  herd immunity     = 1 - 1 / R0
  final size z      solves   z = 1 - exp(-R0 * z)
"""
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import brentq
import matplotlib.pyplot as plt

beta, gamma = 0.5, 0.2
R0 = beta / gamma
print(f"R0 = {R0},  herd immunity threshold = {1 - 1 / R0:.2f}")


# The final-size equation has no closed form, so find its root numerically
# (R: uniroot).
def final_size(R0):
    if R0 <= 1:
        return 0.0
    return brentq(lambda z: z - (1 - np.exp(-R0 * z)), 1e-6, 1)


print(f"final size from the equation = {final_size(R0):.4f}")


# Check against a simulated epidemic
def sir(t, y, beta, gamma, N):
```

```
    S, I, R = y
    return [-beta * S * I / N, beta * S * I / N - gamma * I, gamma * I]


N = 1000
sol = solve_ivp(sir, (0, 300), [N - 1, 1, 0], args=(beta, gamma, N),
                rtol=1e-8, t_eval=np.arange(0, 301))
print(f"final size from the simulation = {sol.y[2, -1] / N:.4f}")

# How the final size depends on R0
print("\n  R0   herd immunity   final size")
for R in [1.2, 1.5, 2, 2.5, 3, 4]:
    print(f"  {R:<4} {1 - 1 / R:>10.3f} {final_size(R):>12.3f}")

R0_grid = np.arange(1.05, 5.01, 0.05)
plt.plot(R0_grid, [final_size(R) for R in R0_grid], label="Final size")
plt.plot(R0_grid, 1 - 1 / R0_grid, "--", label="Herd immunity threshold")
plt.xlabel("R0")
plt.ylabel("Fraction of population")
plt.legend()
plt.show()
# The final size is always larger than the herd immunity threshold:
# the epidemic overshoots because infections continue while I is still large.
```

Vaccination and Interventions

Control measures as changes to the model: starting part of the population immune, scanning coverage levels around the herd immunity threshold, and cutting transmission through distancing.

Control measures change the model's inputs. Vaccination (idealised here as giving full immunity) moves people straight into the R compartment before the outbreak, and measures like distancing reduce β.

```
library(deSolve)

sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

# Run an epidemic where a fraction `vacc` of the population starts immune
run_sir <- function(beta = 0.5, gamma = 0.2, N = 1000, vacc = 0, days = 200) {
  start <- c(S = N - vacc * N - 1, I = 1, R = vacc * N)
  as.data.frame(ode(start, seq(0, days, by = 1), sir,
                    c(beta = beta, gamma = gamma, N = N)))
}
```

Compare vaccination coverage

With $R_0 = 2.5$ the herd immunity threshold is $1 - 1/2.5 = 0.6$.

```
coverage <- c(0, 0.3, 0.5, 0.7)
cols <- c("black", "blue", "darkgreen", "red")

plot(NULL, xlim = c(0, 200), ylim = c(0, 350),
     xlab = "Day", ylab = "Infectious people")
for (i in seq_along(coverage)) {
  out <- run_sir(vacc = coverage[i])
  lines(out$time, out$I, col = cols[i], lwd = 2)
}
legend("topright", paste0(coverage * 100, "% vaccinated"), col = cols, lwd = 2)
```

At 70% coverage (above the 60% threshold) the single introduced case cannot start an epidemic.

How many people are infected at each coverage level?

```
infected_by_coverage <- function(p) {
  out <- run_sir(vacc = p)
  tail(out$R, 1) - p * 1000          # recovered minus those vaccinated
}

p_grid <- seq(0, 0.9, by = 0.05)
total_infected <- sapply(p_grid, infected_by_coverage)

plot(p_grid, total_infected, type = "b", pch = 16,
     xlab = "Vaccination coverage", ylab = "People infected (of 1000)")
abline(v = 0.6, lty = 2)
```

Reducing transmission instead

A 40% cut in contacts lowers β from 0.5 to 0.3, so R_0 falls from 2.5 to 1.5:

```
base  <- run_sir(beta = 0.5)
cut40 <- run_sir(beta = 0.3)
c(peak_baseline = round(max(base$I)), peak_with_distancing = round(max(cut40$I)))
```

Distancing flattens and delays the peak even when it does not stop the epidemic.

▶ View R Markdown

▼ View Python script

○ ○ ○ mathepi/python/04-vaccination.py

```
"""
MathEpi 04 - Vaccination and interventions

Idealised vaccination: a fraction `vacc` of the population starts fully
immune (in R). Distancing: reduce beta.
With R0 = 2.5 the herd immunity threshold is 1 - 1/2.5 = 0.6.
"""
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt


def sir(t, y, beta, gamma, N):
    S, I, R = y
    return [-beta * S * I / N, beta * S * I / N - gamma * I, gamma * I]


def run_sir(beta=0.5, gamma=0.2, N=1000, vacc=0.0, days=200):
    y0 = [N - vacc * N - 1, 1, vacc * N]
    return solve_ivp(sir, (0, days), y0, args=(beta, gamma, N),
                     rtol=1e-8, t_eval=np.arange(0, days + 1))


# Compare vaccination coverage
for p, colour in zip([0, 0.3, 0.5, 0.7], ["black", "blue", "green", "red"]):
    sol = run_sir(vacc=p)
    plt.plot(sol.t, sol.y[1], color=colour, label=f"{int(p * 100)}% vaccinated")
plt.xlabel("Day")
plt.ylabel("Infectious people")
plt.legend()
plt.show()


# How many people are infected at each coverage level?
def infected_by_coverage(p, N=1000):
    sol = run_sir(vacc=p, N=N)
    return sol.y[2, -1] - p * N          # recovered minus those vaccinated


p_grid = np.arange(0, 0.95, 0.05)
total_infected = [infected_by_coverage(p) for p in p_grid]
```

```
plt.plot(p_grid, total_infected, "o-")
plt.axvline(0.6, linestyle="--", color="grey")
plt.xlabel("Vaccination coverage")
plt.ylabel("People infected (of 1000)")
plt.show()

# Reducing transmission instead: a 40% cut in contacts (beta 0.5 -> 0.3)
base = run_sir(beta=0.5)
cut40 = run_sir(beta=0.3)
print(f"peak, baseline       = {base.y[1].max():.0f}")
print(f"peak, with distancing = {cut40.y[1].max():.0f}")
```

The SEIR Model

Adding a latent period with an Exposed compartment. The R_0 and the final size stay the same, but the epidemic is delayed and the infectious peak is lower.

Many infections have a **latent period**: a person is infected but not yet infectious. The SEIR model adds an **E**xposed compartment:

$$\frac{dS}{dt} = -\beta\frac{SI}{N}, \quad \frac{dE}{dt} = \beta\frac{SI}{N} - \sigma E, \quad \frac{dI}{dt} = \sigma E - \gamma I, \quad \frac{dR}{dt} = \gamma I$$

where $\sigma = 1/(\text{latent period})$. Here we use a 5-day latent period ($\sigma = 0.2$).

```
library(deSolve)

seir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dE <-  beta * S * I / N - sigma * E
    dI <-  sigma * E - gamma * I
    dR <-  gamma * I
    list(c(dS, dE, dI, dR))
  })
}

sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

N <- 1000
times <- seq(0, 200, by = 1)

seir_out <- as.data.frame(ode(c(S = N - 1, E = 0, I = 1, R = 0), times, seir,
                              c(beta = 0.5, sigma = 0.2, gamma = 0.2, N = N)))
sir_out  <- as.data.frame(ode(c(S = N - 1, I = 1, R = 0), times, sir,
                              c(beta = 0.5, gamma = 0.2, N = N)))
```

SIR vs SEIR

```
plot(sir_out$time, sir_out$I, type = "l", lwd = 2, col = "red",
     xlab = "Day", ylab = "Infectious people", ylim = c(0, 350))
lines(seir_out$time, seir_out$I, lwd = 2, col = "blue")
legend("topright", c("SIR", "SEIR"), col = c("red", "blue"), lwd = 2)
```

```
data.frame(
  model = c("SIR", "SEIR"),
  peak_day = c(sir_out$time[which.max(sir_out$I)], seir_out$time[which.max(seir_out$I)]),
  peak_infectious = round(c(max(sir_out$I), max(seir_out$I)))
)
```

Both models have the same $R_0 = \beta/\gamma = 2.5$, so they share the same final size, but the latent period **delays** the epidemic and lowers the peak of the infectious curve.

▶ View R Markdown

▼ View Python script

○ ○ ○ mathepi/python/05-seir-model.py

```
"""
MathEpi 05 - The SEIR model
```

```
Adds an Exposed compartment (infected but not yet infectious):

    dS/dt = -beta * S * I / N
    dE/dt =  beta * S * I / N - sigma * E
    dI/dt =  sigma * E - gamma * I
    dR/dt =  gamma * I

sigma = 1 / (latent period).  Here: 5-day latent period, sigma = 0.2.
"""
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt


def seir(t, y, beta, sigma, gamma, N):
    S, E, I, R = y
    dS = -beta * S * I / N
    dE = beta * S * I / N - sigma * E
    dI = sigma * E - gamma * I
    dR = gamma * I
    return [dS, dE, dI, dR]


def sir(t, y, beta, gamma, N):
    S, I, R = y
    return [-beta * S * I / N, beta * S * I / N - gamma * I, gamma * I]


N = 1000
days = np.arange(0, 201)

seir_sol = solve_ivp(seir, (0, 200), [N - 1, 0, 1, 0],
                     args=(0.5, 0.2, 0.2, N), rtol=1e-8, t_eval=days)
sir_sol = solve_ivp(sir, (0, 200), [N - 1, 1, 0],
                    args=(0.5, 0.2, N), rtol=1e-8, t_eval=days)

plt.plot(days, sir_sol.y[1], "r", label="SIR")
plt.plot(days, seir_sol.y[2], "b", label="SEIR")
plt.xlabel("Day")
plt.ylabel("Infectious people")
plt.legend()
plt.show()

print("model   peak_day  peak_infectious")
print(f"SIR     {days[sir_sol.y[1].argmax()]:>8}  {sir_sol.y[1].max():>10.0f}")
print(f"SEIR    {days[seir_sol.y[2].argmax()]:>8}  {seir_sol.y[2].max():>10.0f}")
# Same R0 = beta / gamma = 2.5, so the same final size,
# but the latent period delays the epidemic and lowers the infectious peak.
```

Stochastic Epidemics (Reed-Frost)

When only a few people are infected, chance rules. Run a chain-binomial model

hundreds of times to see minor and major outbreaks, and compare with the extinction probability from branching-process theory.

Differential equations describe the average behaviour of a big population. When only a few people are infected, chance matters: a single case may infect nobody and the outbreak dies out immediately.

The **Reed-Frost** chain-binomial model captures this in discrete generations. If I_t people are infectious and each pair of people has probability p of a transmitting contact, a susceptible escapes infection with probability $(1-p)^{I_t}$. So

$$I_{t+1} \sim \text{Binomial}\big(S_t,\ 1-(1-p)^{I_t}\big).$$

```
reed_frost <- function(N = 200, I0 = 1, p = 0.01) {
  S <- N - I0
  I <- I0
  total <- I0
  while (I > 0) {
    new_I <- rbinom(1, S, 1 - (1 - p)^I)
    S <- S - new_I
    total <- total + new_I
    I <- new_I
  }
  total                             # total number ever infected
}

N <- 200; p <- 0.01
R0 <- (N - 1) * p                   # about 2
R0
```

Run many outbreaks

```
set.seed(42)
sizes <- replicate(500, reed_frost(N = N, p = p))

hist(sizes, breaks = 30, col = "grey80",
     xlab = "Total infected (out of 200)", main = "500 simulated outbreaks")
```

The histogram is **bimodal**: many outbreaks fizzle out early (minor outbreaks), while others take off and infect a large part of the population (major outbreaks).

Probability of a minor outbreak

```
mean(sizes < 20)                    # fraction of outbreaks that fizzled
```

Early in an outbreak, the number of new cases from each case behaves like a branching process. The probability q that it dies out solves $q = e^{R_0(q-1)}$:

```
q <- uniroot(function(q) q - exp(R0 * (q - 1)), c(0, 0.999))$root
q                                   # theory
```

Even with $R_0 \approx 2$, roughly one introduction in five goes nowhere. The deterministic SIR model cannot show this, which is why stochastic models are important for small populations and early outbreaks.

▶ View R Markdown

▼ View Python script

○ ○ ○ mathepi/python/06-stochastic-reed-frost.py

```
"""
MathEpi 06 - Stochastic epidemics (Reed-Frost chain binomial)

With few infected people, chance matters. In discrete generations, if I_t
people are infectious and each pair has probability p of a transmitting
contact, a susceptible escapes with probability (1 - p)**I_t, so

    I_{t+1} ~ Binomial(S_t, 1 - (1 - p)**I_t)
"""
import numpy as np
from scipy.optimize import brentq
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)


def reed_frost(N=200, I0=1, p=0.01):
    S, I, total = N - I0, I0, I0
    while I > 0:
        new_I = rng.binomial(S, 1 - (1 - p) ** I)
        S -= new_I
        total += new_I
        I = new_I
    return total                      # total number ever infected


N, p = 200, 0.01
R0 = (N - 1) * p                      # about 2
print("R0 =", R0)

# Run many outbreaks
sizes = np.array([reed_frost(N=N, p=p) for _ in range(500)])

plt.hist(sizes, bins=30, color="lightgrey", edgecolor="grey")
plt.xlabel("Total infected (out of 200)")
plt.title("500 simulated outbreaks")
plt.show()
# The histogram is bimodal: minor outbreaks fizzle out, major ones take off.

# Probability of a minor outbreak
print("fraction that fizzled (<20 infected):", np.mean(sizes < 20))

# Early on the epidemic behaves like a branching process; the probability q
# that it dies out solves   q = exp(R0 * (q - 1)).
q = brentq(lambda q: q - np.exp(R0 * (q - 1)), 0, 0.999)
```

```
print("theory:", round(q, 3))
# Even with R0 ~ 2, roughly one introduction in five goes nowhere.
```

Fitting a Model to Data

Estimating a transmission rate from data by least squares: simulate noisy observations from a known epidemic, minimise the squared error, and recover β and R_0.

Models become useful when their parameters are estimated from data. Here we generate "observed" data from an SIR epidemic with known parameters, then recover the transmission rate β by **least squares**: choose the β that makes the model curve closest to the observations.

Simulate noisy observations

```
library(deSolve)

sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

N <- 1000
gamma <- 0.2                  # assume the infectious period is known (5 days)
days <- 0:60

model_I <- function(beta) {
  out <- ode(c(S = N - 1, I = 1, R = 0), days, sir,
             c(beta = beta, gamma = gamma, N = N))
  out[, "I"]
}

set.seed(10)
true_beta <- 0.5
observed <- rpois(length(days), model_I(true_beta))   # Poisson observation noise
```

Define and minimise the error

```
sse <- function(beta) sum((observed - model_I(beta))^2)

fit <- optimize(sse, interval = c(0.1, 1))
beta_hat <- fit$minimum
beta_hat                          # close to the true value 0.5
beta_hat / gamma                  # estimated R0 (true value 2.5)
```

Compare the fitted curve with the data

```
plot(days, observed, pch = 16, xlab = "Day", ylab = "Infectious people")
lines(days, model_I(beta_hat), col = "red", lwd = 2)
legend("topright", c("Observed", "Fitted SIR"), pch = c(16, NA),
       lty = c(NA, 1), col = c("black", "red"))
```

See why it works: the error curve

```
beta_grid <- seq(0.3, 0.7, by = 0.01)
plot(beta_grid, sapply(beta_grid, sse), type = "l", lwd = 2,
     xlab = "beta", ylab = "Sum of squared errors")
abline(v = beta_hat, lty = 2, col = "red")
```

The error has a clear minimum near the true β. In a real analysis you would also estimate γ, report uncertainty, and use a likelihood appropriate for count data, but the logic is exactly this.

▶ View R Markdown

▼ View Python script

○ ○ ○ mathepi/python/07-fitting-sir.py

```
"""
MathEpi 07 - Fitting a model to data (least squares)

Generate "observed" data from an SIR epidemic with known parameters, then
recover the transmission rate beta by choosing the value that makes the
model curve closest to the data.
"""
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import minimize_scalar
import matplotlib.pyplot as plt


def sir(t, y, beta, gamma, N):
    S, I, R = y
    return [-beta * S * I / N, beta * S * I / N - gamma * I, gamma * I]


N = 1000
gamma = 0.2                     # assume the infectious period is known (5 days)
days = np.arange(0, 61)


def model_I(beta):
    sol = solve_ivp(sir, (0, 60), [N - 1, 1, 0], args=(beta, gamma, N), rtol=1e-8, t_eval=days)
    return sol.y[1]


# Simulate noisy observations (Poisson noise around the true curve)
```

```
rng = np.random.default_rng(10)
true_beta = 0.5
observed = rng.poisson(model_I(true_beta))


# Least squares: pick the beta that minimises the sum of squared errors
def sse(beta):
    return np.sum((observed - model_I(beta)) ** 2)


fit = minimize_scalar(sse, bounds=(0.1, 1.0), method="bounded")
beta_hat = fit.x
print(f"estimated beta = {beta_hat:.4f}   (true {true_beta})")
print(f"estimated R0   = {beta_hat / gamma:.3f}   (true 2.5)")

plt.plot(days, observed, "ko", label="Observed")
plt.plot(days, model_I(beta_hat), "r-", label="Fitted SIR")
plt.xlabel("Day")
plt.ylabel("Infectious people")
plt.legend()
plt.show()

# The error curve has a clear minimum near the true beta
beta_grid = np.arange(0.3, 0.701, 0.01)
plt.plot(beta_grid, [sse(b) for b in beta_grid])
plt.axvline(beta_hat, linestyle="--", color="red")
plt.xlabel("beta")
plt.ylabel("Sum of squared errors")
plt.show()
```


AIMS | African Institute for Mathematical Sciences RESEARCH & INNOVATION

Dr. Lema Logamou Seknewna

AIMS-NISR Data Scientist, Researcher and Lecturer

African Institute for Mathematical Sciences, Research and Innovation Centre